

A Survey on the Key Technologies of UAV Motion Planning

Yuquan Zhou ^{1,†} , Li Yan ^{1,†}, Yaxi Han ¹, Hong Xie ^{1,*} and Yinghao Zhao ^{2,*} 

¹ School of Geodesy and Geomatics, Hubei LuoJia Laboratory, Wuhan University, Wuhan 430079, China; zhouyuquan@whu.edu.cn (Y.Z.); liyan@sgg.whu.edu.cn (L.Y.); whuhyx@whu.edu.cn (Y.H.)

² School of Geospatial Information, Information Engineering University, Zhengzhou 450002, China

* Correspondence: hxie@sgg.whu.edu.cn (H.X.); zyhiu@163.com (Y.Z.)

† These authors contributed equally to this work.

Abstract: Unmanned aerial vehicles (UAVs) are widely employed across diverse fields due to their flexibility and scalability. However, achieving full autonomy remains a challenge as human intervention is still required in most scenarios. Motion planning, a cornerstone of UAV autonomous navigation, has garnered extensive attention, with numerous advanced algorithms having been proposed in recent years. This paper provides a comprehensive overview of UAV motion planning frameworks, systematically addressing three key components: map representation, path planning, and trajectory optimization. Map representation establishes environmental awareness, path planning balances efficiency and safety in path generation, and trajectory optimization refines paths into feasible, energy-efficient motions. Unlike prior reviews focused on specific techniques, this study offers an integrated perspective, helping researchers understand the overall framework and recent advancements in UAV motion planning. Additionally, emerging trends and potential strategies are discussed to improve the efficiency, adaptability, and robustness of UAVs to meet increasingly complex mission requirements.

Keywords: unmanned aerial vehicle; motion planning; map representation; path planning; trajectory optimization



Academic Editor: Andrey V. Savkin

Received: 12 January 2025

Revised: 28 February 2025

Accepted: 4 March 2025

Published: 6 March 2025

Citation: Zhou, Y.; Yan, L.; Han, Y.; Xie, H.; Zhao, Y. A Survey on the Key Technologies of UAV Motion Planning. *Drones* **2025**, *9*, 194. <https://doi.org/10.3390/drones9030194>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Unmanned aerial vehicles (UAVs) have garnered significant attention due to their high flexibility and adaptability, and they have been successfully deployed in areas such as surveying and mapping [1–3], 3D reconstruction [4–6], search-and-rescue [7–10], precision agriculture [11–13], etc., demonstrating notable effectiveness. However, current UAV operation modes are still predominantly limited to manual control or pre-programmed routines, which provide limited autonomy and intelligence. This limitation makes it challenging for UAVs to fulfill autonomous operational requirements in complex and unknown environments, such as under canopies [14–16], indoor [17–20], urban [21–24], subterranean [25–27], etc. Consequently, the development of intelligent UAVs with autonomous navigation capabilities has become a key focus in the field of robotics research.

As the core component of autonomous navigation, motion planning technology has undergone extensive research and development in recent years (Figure 1 presents the number of published articles on UAV motion planning from 2015 to 2024). These works were able to generate an optimal or sub-optimal path, depending on the actual task requirements, within either static or dynamic operational environments. They considered key criteria such as safety, smoothness, and energy efficiency while adhering to dynamic constraints, ensuring that the robot travels from its current state to its target state without human

intervention. However, different motion planning algorithms have been tailored to distinct application scenarios, each with their unique strengths and limitations. Therefore, it is necessary to summarize and categorize these algorithms to facilitate efficient learning and application for researchers interested in the current advancements in motion planning.

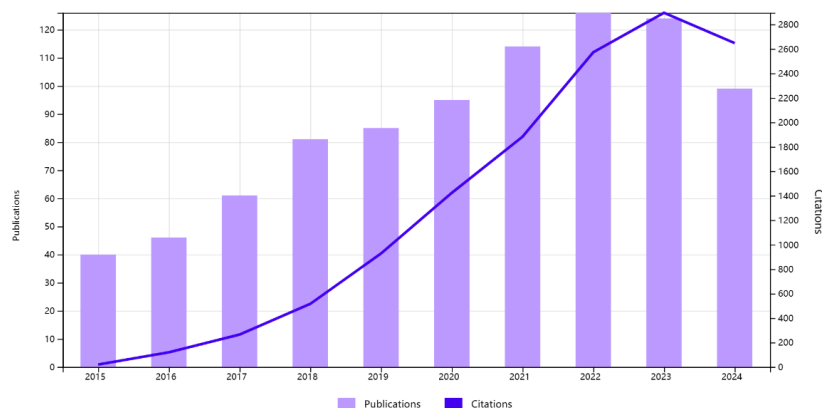


Figure 1. The publications and citations in the field of UAV motion planning have shown a noticeable trend from 2015 to 2024 (based on data sourced from Web of Science).

Motion planning for UAVs consists of three components (as shown in Figure 2): map representation, path planning, and trajectory optimization. Map representation involves the use of onboard sensors to perceive the UAV's surrounding environment in real time, which is then abstracted into a computational form that is suitable for storage and processing. Path planning, in contrast, involves quickly identifying a collision-free geometric path between the start and goal points by performing searches or utilizing a sampling manner based on the map representation. The trajectory optimization module then refines this initial path using metrics such as smoothness, safety, and dynamic feasibility to produce a motion trajectory that is safe, smooth, and executable by the UAV. To the best of our knowledge, many existing review articles in the UAV motion planning domain usually focus on only one key module, such as path planning [28–30] or trajectory optimization [31–33]. It is clear that these three components are closely interrelated, and examining only one of them may result in an incomplete understanding of the overall UAV motion planning framework.

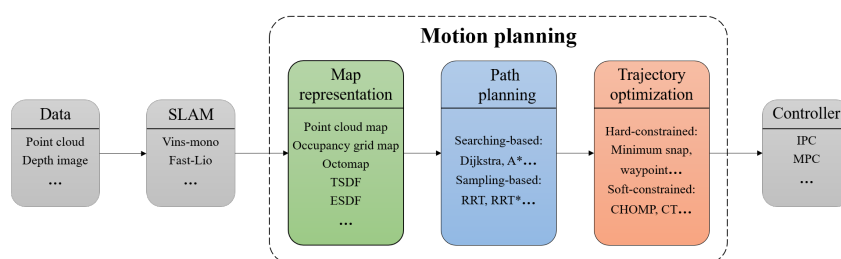


Figure 2. Overview of the UAV motion planning pipeline.

To this end, we present a systematic overview of the UAV motion planning workflow, focusing on three key components: map representation, path planning, and trajectory optimization. This study aims to provide researchers with a comprehensive understanding of the motion planning framework by analyzing both the classical works and recent advancements in these modules. By doing so, we seek to address the challenges of enhancing UAV autonomy and adaptability in increasingly complex operational environments. Notably, we exclude discussions on topics such as simultaneous localization and mapping (SLAM), control strategies, and learning-based approaches. Furthermore, we review representative UAV motion planning frameworks that have been validated in real-world environments

over the past four years, categorizing them based on the aforementioned three components. This structured analysis highlights the unique features, strengths, and limitations of these algorithms, providing valuable insights to guide future advancements. Finally, we summarize the current state of UAV motion planning development and propose forward-looking directions for addressing the limitations of current methods.

The remainder of this paper is organized as follows: Section 2 introduces map representation techniques. Section 3 reviews path planning algorithms. Section 4 discusses trajectory optimization methods. Section 5 examines UAV motion planning frameworks that were validated in real-world experiments developed over the past four years, and it also presents several case studies to demonstrate the performance of these key technologies. Section 6 discusses challenges and future research directions, while Section 7 concludes the paper.

2. Map Representation

Map representation forms the foundation of motion planning, allowing robots to effectively perceive obstacles in unknown environments. This perception enables essential tasks such as path planning and trajectory optimization, which support autonomous navigation. UAV-related map representation technologies, as applied to one of robotics' most prominent platforms, have been widely researched. UAVs, with their high speeds and substantial degrees of freedom, are primarily deployed in unstructured outdoor environments. As a result, UAV motion planning often requires dense 3D maps to accurately represent obstacles like trees and buildings, ensuring safe navigation. However, due to UAVs' typically small size and limited endurance, their computational resources are constrained. Overly detailed map representations can increase memory consumption and computational load, potentially impeding real-time processing. Therefore, current UAV map representations strike a careful balance between accuracy and computational efficiency. The primary map representations for UAV motion planning include the point cloud map (PCM) [34], occupancy grid map (OGM) [35], Octomap [36], truncated signed distance field (TSDF) [37], Euclidean signed distance field (ESDF) [38], and semantic map (SM) [39]. The practical performance of various map representations is illustrated in Figure 3.

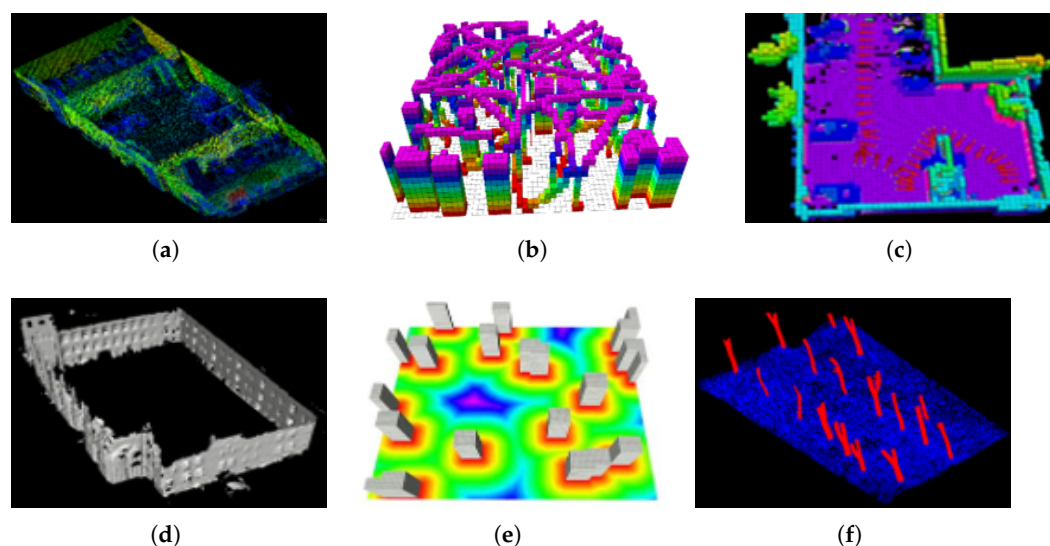


Figure 3. Several commonly used map representations in UAV motion planning. (a) PCM [34]. (b) OGM [36]. (c) Octomap [40]. (d) TSDF [41]. (e) ESDF [42]. (f) SM [43].

PCMs [44] are composed of a series of points in 3D space acquired by sensors such as LiDAR or depth cameras. These maps require no additional post-processing and can adapt

well to various types of sensors and environments. However, querying such large amounts of unstructured data can be very time consuming. To address this issue, ref. [34] employed a KD-tree [45] data structure for efficient map storage and rapid nearest neighbor searches, enabling real-time motion planning directly within PCMs.

OGMs were initially proposed by ref. [35], discretizing the robot's configuration space into 2D or 3D grids, where each cell belongs to one of three states: free, occupied, or unknown. It is easy to query; however, memory consumption grows rapidly with increasing map resolution. Additionally, traditional OGMs treat each grid cell as a one-dimensional estimation problem, ignoring the spatial continuity between cells, which can lead to inaccurate mapping. Ref. [40] employs an explicit representation of all three states in the map, utilizing Morton codes to effectively reduce memory overhead and accelerate query efficiency. Moreover, ref. [46] introduced an OGM based on 3D region-wise hashing, enabling online dynamic obstacle removal. In [47], LiDAR was combined with an OGM to improve query efficiency and reduce memory costs in large-scale scenarios by maintaining a sliding local map centered on the UAV.

Octomap was developed by ref. [36] based on octree encoding [48]. Octomap recursively subdivides 3D space into smaller cubes, represented as nodes in an octree, enabling efficient storage of information at varying levels of detail. However, maintaining an up-to-date Octomap requires constant updates, leading to high computational overhead. To address this, ref. [49] introduced a circular buffer for real-time localized map construction, significantly reducing memory usage and computational overhead. In contrast, ref. [50] proposed an efficient accelerator solution called OMU for real-time probabilistic 3D mapping, including parallel updates and optimized in-memory data storage.

The TSDF was introduced by [37], which defines the distance value from each voxel to the surface of an obstacle within a specific range, and the distance values beyond this range are truncated to reduce the computational effort. The TSDF can efficiently represent complex environments, is robust to sensor noise, and can be updated in real time. Ref. [41] proposed a method for quadcopter UAVs with an online planning scheme that uses OGM and TSDF as map representation and achieves autonomous navigation in unknown indoor and outdoor environments.

The ESDF was introduced by [38] to define the minimum distance from each unoccupied voxel in space to the nearest obstacle voxel, making it highly suitable for evaluating the potential cost of obstacle collisions. ESDF has garnered significant attention in the field of UAV motion planning since its use as a map representation in CHOMP [51]. Ref. [52] proposed Voxblox, which incrementally builds ESDF from TSDF, improving the speed of ESDF construction. However, the final ESDF produced by Voxblox may still have significant distance errors. To address this, ref. [42] proposed FIESTA, a lightweight and flexible mapping framework that incrementally constructs ESDF from any base map, achieving faster and more accurate results compared to Voxblox [52]. In contrast, ref. [53] introduced a robot-centered ESDF that pre-builds within the robot body frame, considering only obstacle points that fall within the field of view, significantly reducing computational overhead. Similar to [52], ref. [54] also builds the ESDF incrementally from TSDF but proposes a novel weighting function to update the ESDF instead of directly projecting the TSDF, resulting in greater accuracy compared to Voxblox[52].

SMs [39] not only represent the spatial layout of the environment, but also incorporate the semantic information about objects and regions in the environment. By incorporating semantic information into maps, robots can better understand their surroundings and make more informed decisions about how to interact with the environment. Ref. [55] applied semantic maps as map representations for the first time in the field of UAV motion planning, realizing the autonomous navigation of UAVs in unknown environments. Ref. [43] devel-

oped a semantic closed-loop (SLC) detection module to enhance localization and mapping accuracy. Additionally, a framework was proposed to balance exploration and exploitation by leveraging this semantic map, effectively reducing localization uncertainty while ensuring efficient exploration. Table 1 summarizes the commonly used map representations in UAV motion planning and their respective advantages and limitations.

Table 1. Several commonly used map representations and their advantages and limitations.

Ref.	Map Representation	Advantages	Limitations
[34]	PCM	• Easy to build • Flexibility of data acquisition	• Slow inquiry speed • Excessive noise
[35]	OGM	• Fast computation • Fine-grained map	• Memory consumption
[36]	Octomap	• Memory efficient • Fast inquiry speed	• Maintenance expensive • Computationally expensive
[37]	TSDF	• Update in real time • Robust to noise	• Computationally expensive • Artifacts and distortions
[38]	ESDF	• Abundant gradient information • Robust to noise	• Computationally expensive • Large memory consumption
[39]	SM	• Memory efficient • Less ambiguous	• Computationally expensive • Limited adaptability

3. Path Planning

Path planning is a critical component of UAV motion planning, where the obstacle information from map representations is utilized to quickly determine a collision-free geometric path. This path serves as an initial solution, which the trajectory optimization module subsequently refines into a safe, smooth, and executable trajectory for the UAV. The quality of the initial path directly impacts the computational load on the trajectory optimization module and influences the quality of the final trajectory. While trajectory optimization can improve aspects such as the path's shape, smoothness, and dynamic feasibility, it cannot fundamentally alter the overall direction or topology. As such, an efficient and reliable path planning module is essential for effective motion planning. Researchers have developed a variety of path planning algorithms, broadly categorized into searching-based and sampling-based approaches. Figure 4 features prominent examples of each category.

3.1. Searching-Based Path Planning

As a category of traditional path planning methods, searching-based methods employ predefined rules to discretize high-dimensional state spaces, transforming the continuous search problem into a graph search problem and significantly reducing computational complexity. Depth-first search (DFS) and breadth-first search (BFS) are two classic types of searching-based methods [56]. DFS uses a last-in, first-out stack structure, enabling it to quickly find a feasible path, although it does not guarantee optimality. Conversely, BFS follows a first-in, first-out queue structure, ensuring path optimality but at a higher computational cost. While both DFS and BFS can efficiently find an initial path in unweighted graphs, such algorithms are inadequate for UAVs, which require efficient path planning in complex environments. In UAV applications, factors such as obstacle avoidance, en-

energy consumption, and dynamic environmental conditions must be considered, which are difficult to address with basic search algorithms. This limitation necessitates the use of weighted graph search algorithms to meet UAV-specific requirements.

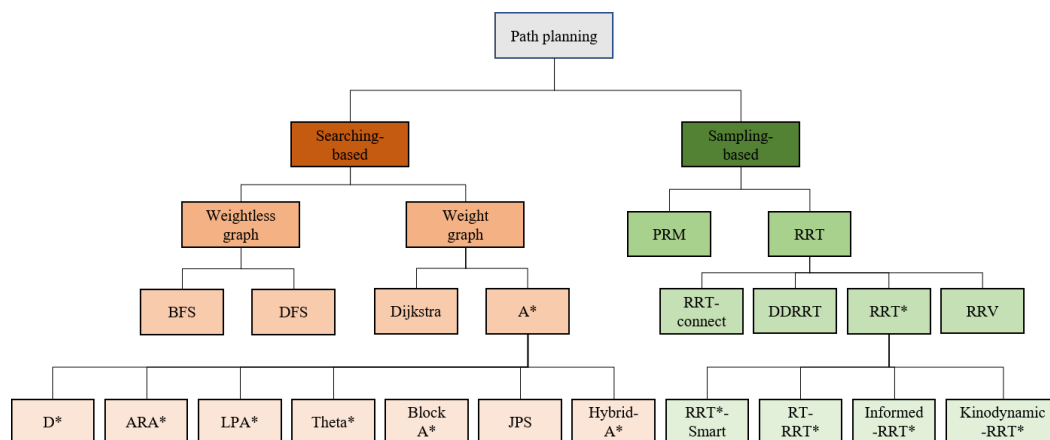


Figure 4. Classification of path planning algorithms.

For weighted graphs, several sophisticated algorithms have been developed, including the well-known Dijkstra’s algorithm [57]. Dijkstra’s algorithm, a greedy approach, guarantees an optimal path from the start to the goal, provided such a path exists. We evaluated the performance of the Dijkstra path planning algorithm through simulations conducted using online tools [58]. The results of the Dijkstra on a 2D grid map are illustrated in Figure 5, where the green and red boxes represent the start and goal points; the gray boxes indicate obstacles; the light blue and light green boxes mark nodes that were expanded during path planning; and the yellow line segments trace the resulting path. Dijkstra’s algorithm functions by expanding the node with the lowest cumulative cost from the start node at each step. However, as not every expanded node contributes to the final path, the computational time grows significantly as the environment’s complexity increases. (The searching-based path planning methods illustrated in Figures 6–8 were generated using the tool, whereas Figure 9 provides a illustrative representation.)

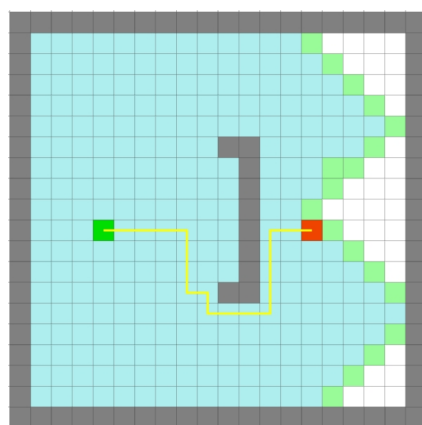


Figure 5. The result of the Dijkstra path planning algorithm in a 2D grid map.

In order to improve the search efficiency of Dijkstra’s algorithm in large and complex environments, A* [59] was proposed. By adding a heuristic function to Dijkstra’s framework, A* significantly accelerates convergence in large and complex environments. The results of the A* path planning algorithm on a 2D grid map are shown in Figure 6 (all of the elements in Figure 6 retain the same meaning as in Figure 5). The introduction of a heuristic function in A* significantly reduces the number of nodes, which need to be expanded to

find the optimal path compared to Dijkstra's algorithm, as shown by the reduced number of light blue and light green boxes in Figure 6. Due to its simplicity and effectiveness, A* has garnered considerable attention, leading to the development of numerous variants over the decades.

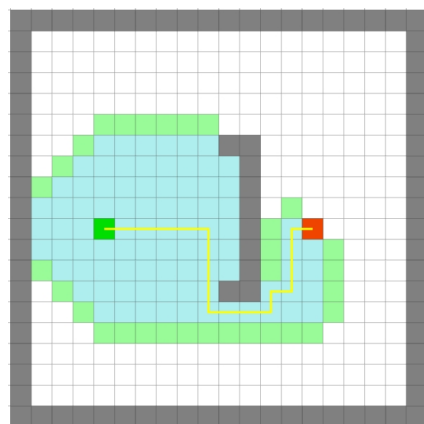


Figure 6. The result of the A* path planning algorithm in a 2D grid map.

While A* offers fast convergence, it requires a full re-search of the path when navigating in dynamic environments, which significantly increases computation time. To address this, ref. [60] introduced D*, an incremental search method for path planning in dynamic environments. Unlike traditional A*, D* only recalculates the affected areas if new obstacles arise during the robot's movement rather than replanning the entire path, thus saving computation time. However, in complex environments, these methods may still take considerable time to find an initial feasible solution. To tackle this, ref. [61] proposed ARA*, an incremental search approach that combines heuristic and incremental search, making it ideal for scenarios requiring a feasible path within a limited time frame, with gradual improvements in path quality. LPA* [62] is another incremental variant of A*, and it is also capable of swiftly updating affected path segments in dynamic environments and, compared to D*, is better suited for highly dynamic settings.

However, the paths generated by the above methods typically consist of a series of grid nodes, resulting in low path smoothness. To address this, ref. [63] proposed Theta*, an any-angle variant of A*, which optimizes the path through "line-of-sight" checks, reducing sharp turns and unnecessary bends to produce smoother paths than traditional A*. An example of the path generated by Theta* in a 2D grid map is shown in Figure 7. Similar to Theta*, Block A* [64] is another any-angle variant of A*, and it consolidates adjacent obstacles into a single entity through a "blocking" mechanism. This approach reduces the number of nodes that need to be explored, thereby lowering computational costs and enhancing performance, especially in environments dense with obstacles.

Although many of the aforementioned A* variants have shown improvements in real-time performance, adaptability to different scenes, and path planning effectiveness, they still tend to expend a large number of nodes that do not contribute to the final path. This leads to increased memory and computational burden. To tackle this problem, ref. [65] introduced a fast version of A* named JPS. It selectively examines specific neighbors, termed "jump points", based on predefined rules. When a neighbor meets the criteria of a "jump point", the algorithm bypasses intermediate nodes and jumps directly to this point. This targeted approach substantially reduces the number of nodes accessed, enhancing both memory and computational efficiency. As shown in Figure 8, JPS expands significantly fewer nodes than the basic A* algorithm to find the optimal path. The light gray boxes indicate the nodes ignored by JPS during the path planning process.

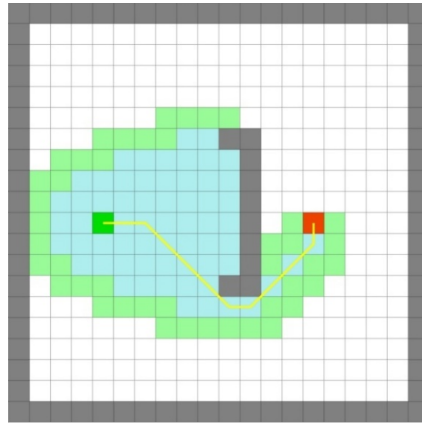


Figure 7. Schematic diagram of the path planning algorithm based on Theta*.

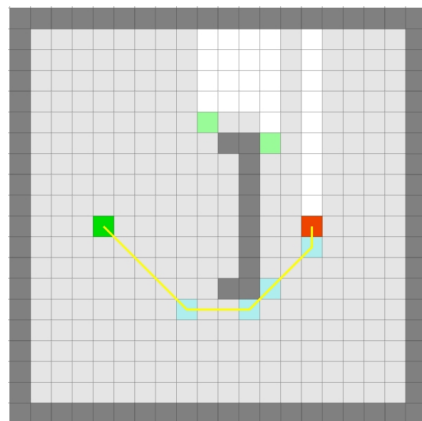


Figure 8. Schematic diagram of the path planning algorithm based on JPS.

Although some any-angle algorithms, such as Theta* [63] and Block A* [64], improve path smoothness by minimizing corners and unnecessary turns, the resulting paths remain line segments paths without any dynamic information. This limitation prevents the paths from being directly executable by robots and inadvertently increases the burden on the back-end trajectory optimization module. To this end, Hybrid A* [66] combines A* with a state lattice [67] to enable path planning that incorporates robot dynamic constraints, such as maximum velocity and acceleration. Consequently, paths generated by Hybrid A* inherently meet certain dynamic feasibility requirements, helping to reduce the computational load on the back-end trajectory optimization module and increasing the likelihood of successful optimization. The result of the Hybrid A* algorithm search in an initial path in a 2D grid map is demonstrated in Figure 9. The arcs indicated by dashed black curves represent the generated motion primitives, while the red curves illustrate the final generated paths.

Overall, searching-based path planning methods provide optimality and completeness, and the incorporation of heuristic search functions helps accelerate algorithm convergence. However, designing effective heuristic functions is essential for different environments and task requirements. To meet real-time computation demands, some paths may compromise optimality and completeness to ensure rapid search results within a limited timeframe. Additionally, in large, complex environments, searching-based algorithms often expand numerous nodes that do not contribute to the final path, leading to excessive memory and computational resource consumption.

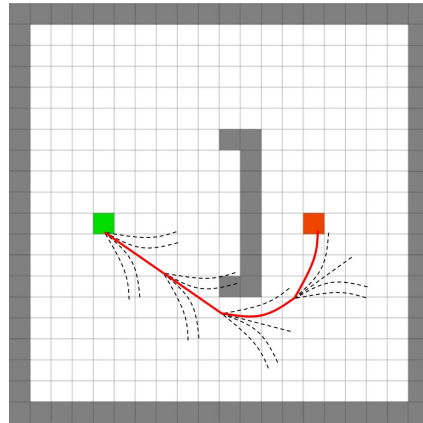


Figure 9. Schematic diagram of the path planning algorithm based on Hybrid A*.

3.2. Sampling-Based Path Planning

To address the limitations of searching-based path planning methods in large-scale scenarios, sampling-based methods have been developed. Probabilistic roadmap (PRM) [68] is a well-established sampling-based path planning algorithm, and it is illustrated in Figure 10 (The sampling-based path planning methods, as shown in Figures 10–15, are all illustrative representations). This algorithm involves two main phases: learning and querying. In the learning phase, as shown in Figure 10a, PRM constructs a roadmap by randomly sampling nodes (light blue dots) in the configuration space and connecting neighboring nodes that are obstacle-free (black line segments). During the querying phase, as depicted in Figure 10b, PRM performs a graph search (such as Dijkstra [57] or A* [59]) on this roadmap to find a collision-free path from the start to the goal (red line segments). Compared to searching-based approaches, PRM lowers memory usage and computational demands.

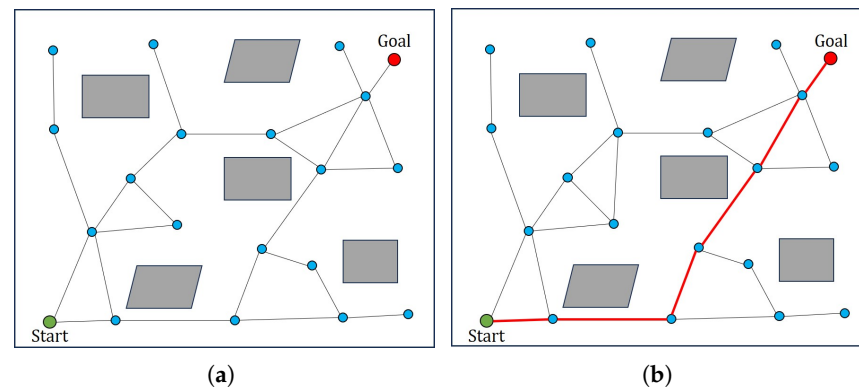


Figure 10. Schematic diagram of the path planning algorithm based on PRM. (a) Roadmap constructed by PRM in the learning phase. (b) Collision-free path searched by PRM on the constructed roadmap during the query phase.

However, due to its unguided random sampling and focus on constructing state space representations rather than directly finding paths, PRM is less suited for real-time path planning in unknown environments. To address these limitations, the rapidly exploring random tree (RRT) algorithm was introduced in [69]. Unlike PRM [68], RRT operates in a single stage and is inherently more goal-oriented, facilitating faster and more direct path planning in unknown environments. Figure 11 illustrates the path planning process of the RRT algorithm in a 2D unknown environment. RRT begins by randomly sampling a node x_{rand} in free space, which is represented by light blue dots in Figure 11a. The algorithm identifies the nearest node x_{near} on the expanding tree to this sampled node, and it then extends a short segment from x_{near} toward x_{rand} to create a new node x_{new} . If the path

between x_{near} and x_{new} is collision-free, x_{new} is added to the tree. This process repeats until the newly generated node x_{new} is sufficiently close to the goal, at which point the algorithm stops iterating. By backtracking through the tree, a collision-free path is obtained from the start to the goal, as shown by the red line segments in Figure 11b.

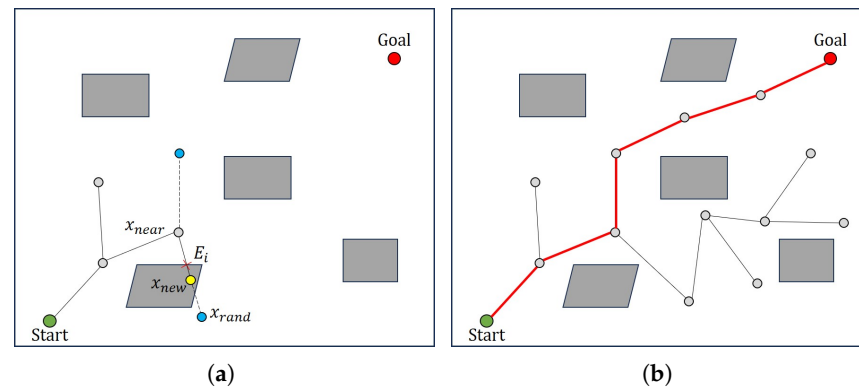


Figure 11. Schematic diagram of the path planning algorithm based on RRT. (a) Incremental construction of the RRT by sampling random nodes and extending collision-free segments. (b) Collision-free path obtained by backtracking through the constructed RRT.

However, RRT has several drawbacks. Due to the randomness of sampling, it struggles to efficiently find paths through narrow passages in the environment. Additionally, the paths generated by RRT are often suboptimal, and directly optimizing these paths may lead to low-quality trajectories or even optimization failure. To overcome these challenges, numerous RRT-based variants have been developed. To improve the convergence speed of the RRT algorithm, RRT-Connect was introduced by [70]. This enhanced variant of RRT initiates two trees that expand concurrently—one starting from the start and the other from the goal. As each tree extends, it grows randomly into the unknown space, expanding nodes and exploring possible paths through the environment, as illustrated in Figure 12a. When the two trees meet or overlap, a continuous, collision-free path is formed that connects the start and goal points, as shown in Figure 12b. This approach significantly reduces search time as it allows both trees to share exploration effort, effectively halving the distance each tree needs to cover to achieve a connection.

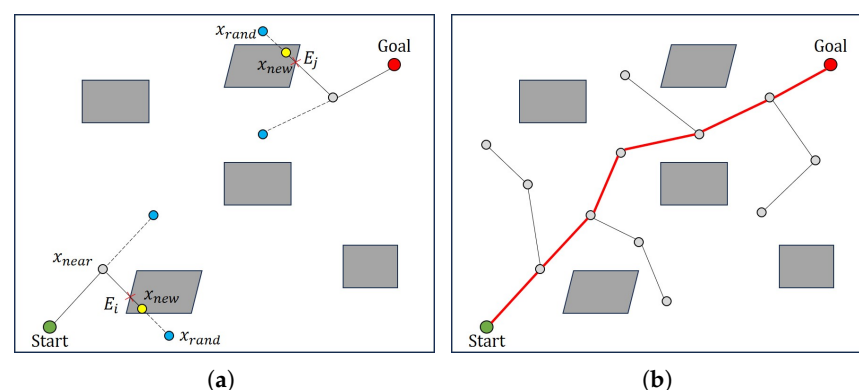


Figure 12. Schematic diagram of the RRT-Connect path planning algorithm. (a) Two trees simultaneously expand from the start and goal positions, incrementally extending towards each other. (b) Collision-free path generated by connecting the two trees when they meet.

Another approach, Dynamic-Domain RRT (DD-RRT) [71], restricts the sampling area to a sphere with a predefined radius, effectively reducing the number of unnecessary nodes and improving the search success rate. In environments with narrow passages, standard

sampling-based approaches often struggle as few sampling points fall within the confined channel. To address this, rapidly exploring random vine (RRV) [72] was proposed. Unlike RRT's tree structure, RRV creates a vine-like structure, allowing it to navigate narrow passages more efficiently. Additionally, RRV incorporates heuristic functions to guide growth toward the target, and it uses local optimization techniques to refine the path while avoiding obstacles. These features make RRV advantageous over RRT in complex environments as it can tackle highly constrained motion planning problems and maneuver flexibly around obstacles.

To improve the quality of the final path produced by RRT, ref. [73] introduced RRT*, an incremental version of RRT that “rewires” the tree structure to minimize overall path cost. This pruning process adjusts neighboring nodes and edges to reduce path costs and enhance the tree's quality. As a result, RRT* satisfies asymptotic optimality, meaning it converges to the optimal path as the iteration count approaches infinity. Figure 13 illustrates the path planning process of the RRT* algorithm in a 2D environment. In Figure 13a, a node x_{new} to be expanded is generated in free space (indicated by the light yellow dot). The algorithm then evaluates nearby nodes within a predefined radius (within the dashed circle in Figure 13b) to determine if any of them can achieve a shorter collision-free path by connecting through x_{new} back to the start. If such a path exists, the parent of these nodes is updated to x_{new} , as shown by the new connection between x_2 and x_{new} in Figure 13b. Once the tree reaches a node sufficiently close to the goal, the algorithm backtracks to generate the final collision-free path, as represented by the red line segments in Figure 13c, resulting in a high-quality initial path for subsequent trajectory optimization.

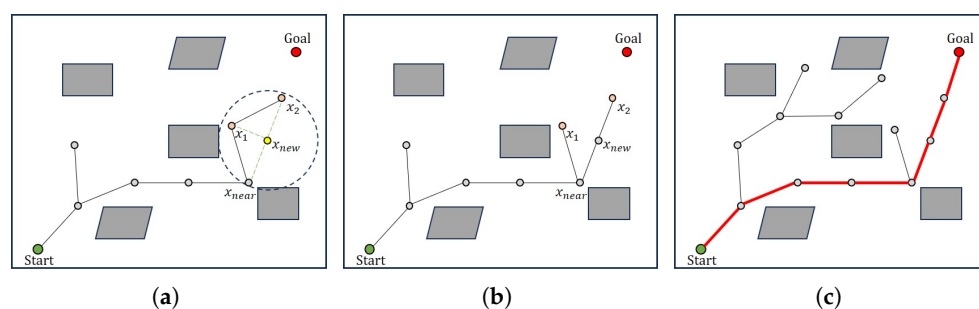


Figure 13. Workflow of path planning algorithm RRT*. For brevity, the random sampling process has been omitted in this illustration. (a) A new node is generated and nearby nodes within a predefined radius are identified. (b) The algorithm evaluates and rewires connections to minimize path cost. (c) Final collision-free optimal path obtained by backtracking through the rewired tree.

RRT* tends to sample many nodes that do not contribute to the final path, resulting in additional memory and computational overhead. To overcome this problem, RRT*-Smart [74] further improves upon RRT* by combining heuristics and optimization techniques. It biases sampling toward regions likely to contain feasible paths and limits the number of pruned nodes per iteration. These refinements make RRT*-Smart more efficient than RRT*, particularly in high-dimensional or complex environments.

The above methods are designed primarily for static environments, making them insufficient for handling the challenges posed by dynamic scenarios. To overcome this limitation, ref. [75] introduced Real time-RRT* (RT-RRT*), a real-time variant of RRT* designed to dynamically adjust planned trajectories in response to environmental changes. Unlike the original RRT*, RT-RRT* continuously updates its tree structure and replans the path with each new sensor data input. Heuristic functions guide the tree expansion toward the goal, taking into account both the current and projected positions of dynamic obstacles. Additionally, a safety buffer is integrated to prevent collisions, making RT-RRT* well suited for path planning in dynamic environments.

To expedite the path optimization following the finding of an initial solution, Informed-RRT* enhances RRT* through using informed sampling [76]. After establishing an initial path, Informed-RRT* limits the sampling region to an ellipsoidal subset shaped by the existing path. As the path shortens, this ellipsoid gradually contracts, focusing sampling efforts on areas more likely to improve the current solution. Figure 14 illustrates the path planning process of the Informed-RRT* algorithm. Figure 14a–c illustrate the progressive optimization process of the path (represented by the red line segments) as the sampling area gradually shrinks (indicated by the dashed ellipses).

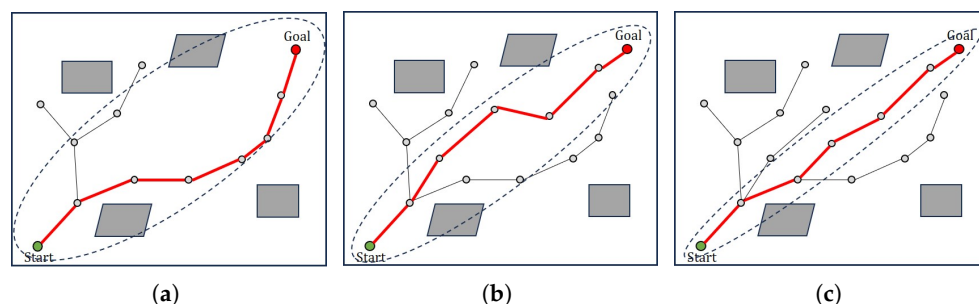


Figure 14. Workflow of the Informed-RRT* path planning algorithm. (a–c) Informed sampling process: the ellipsoidal sampling region (dashed ellipse) gradually shrinks around the current path, focusing sampling efforts to efficiently improve the path quality toward optimality.

However, the aforementioned methods perform sampling without accounting for the robot's dynamic constraints, connecting nodes solely with line segments. Consequently, the resulting paths are typically segmented geometric trajectories, which, while collision-free, are challenging for the robot to execute directly. This limitation places a significant burden on the back-end trajectory optimization module in refining these paths into dynamically feasible trajectories. To address this issue, Kinodynamic-RRT* [77] further extends RRT* by sampling within a kinodynamic state space that includes both velocity and acceleration, ensuring that the resulting path adheres to dynamic constraints. The path planning process of the Kinodynamic-RRT* algorithm is demonstrated in Figure 15. As can be seen, the main process of Kinodynamic-RRT* is similar to that of RRT* but its nodes are connected by smoother curves, making the paths more executable for the robot and more suitable for refinement by the subsequent trajectory optimization module.

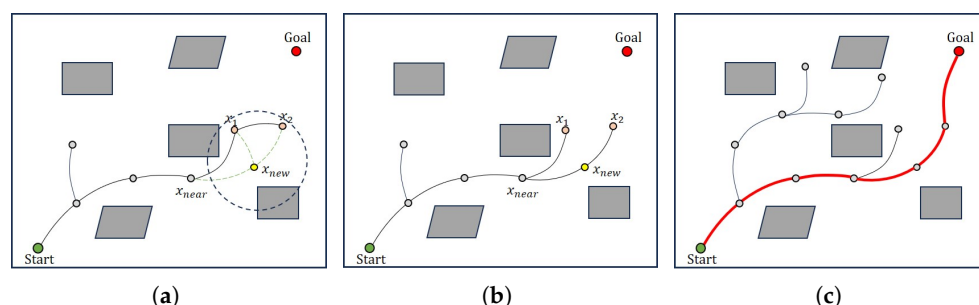


Figure 15. Workflow of the Kinodynamic-RRT* path planning algorithm. (a) Generating new nodes through sampling in state space, considering kinodynamic constraints and identifying neighboring nodes within a predefined radius. (b) Evaluating and selecting optimal connections among candidate nodes using dynamically feasible and path costs to incrementally extend the tree. (c) Obtaining the final smooth, dynamically feasible, and collision-free path by backtracking through the optimized tree structure.

In summary, sampling-based path planning algorithms are effective in reducing memory usage and computational demands in high-dimensional path planning by sampling

nodes randomly across the space. This efficiency makes them well suited for real-time path planning on platforms with limited onboard resources. However, due to their inherent randomness, achieving globally optimal paths remains challenging. Although asymptotically optimal algorithms have been developed, these methods require extensive sampling, which can greatly hinder convergence speed and may not meet the real-time requirements of UAV motion planning. Furthermore, sampling-based methods can struggle with narrow passages in complex environments; although specific adaptations have been proposed, these are often tailored to particular settings, and their robustness in real-world applications has yet to be fully established.

3.3. Discussion

In order to present the performance of various types of path planning algorithms more clearly, Table 2 provides an overview of the advantages and limitations of various classical path planning algorithms. Searching-based methods offer clear benefits in terms of optimality and completeness, especially with the integration of heuristic functions, which accelerate convergence. However, these methods generally demand substantial memory and computational resources, limiting their scalability in large environments. In contrast, sampling-based methods are more efficient in large-scale scenarios, and some asymptotically optimal variants can achieve optimal solutions when sufficient computational resources are available. Despite these strengths, sampling-based methods often face challenges in complex settings with narrow passages.

Table 2. Several classical path-finding algorithms and their advantages and limitations.

Category	Ref.	Method	Advantages	Limitations
Searching-based	[56]	BFS	• Optimality	• Memory intensive • Computationally expensive • Weightless graph only
	[56]	DFS	• Memory efficient • Rapid convergence	• Sub-optimal • Weightless graph only
	[57]	Dijkstra	• Optimality • Completeness	• Uninformed search
	[59]	A*	• Heuristic search • Optimality	• Admissible heuristic function
	[60]	D*	• Incremental search	• Computationally expensive
	[61]	ARA*	• Anytime algorithm • Computationally efficient	• Memory intensive
	[62]	LPA*	• Dynamic environment • Anytime algorithm	• Memory intensive • Static start and end point
	[63]	Theta*	• Smoother path	• Computationally expensive
	[64]	Block A*	• Expand fewer nodes • Any-angle algorithm	• Sensitive to the block size
	[65]	JPS	• Rapid convergence • Memory efficient	• Uniform-cost grid only
	[66]	Hybrid A*	• Kinodynamic feasibility	• Not globally optimal • Not completeness

Table 2. Cont.

Category	Ref.	Method	Advantages	Limitations
Sampling-based	[68]	RPM	• Probabilistically complete	• Computationally expensive • Two-stage planning
	[69]	RRT	• More target-oriented than RPM	• Suboptimality • Sample in the whole space
	[70]	RRT-connect	• Bidirectional search	• Implementation complexity • May not find a path
	[71]	DD-RRT	• Adaptive sampling domain • Dynamic sampling radius	• Ignores samples far from the current node
	[72]	RRV	• High robustness	• Environmental dependence
	[73]	RRT*	• Asymptotic optimality	• Computationally expensive • Memory intensive
	[74]	RRT*-Smart	• Intelligent sampling • Rapid convergence	• Heuristic admissible • Low adaptability
	[75]	RT-RRT*	• Real time • Dynamic environment	• Limited applicability on simple scenarios
	[76]	Informed-RRT*	• Direct sampling • Rapid convergence	• Relies on initial path
	[77]	Kinodynamic-RRT*	• Dynamic feasibility	• Computationally expensive

In summary, both searching-based and sampling-based path planning algorithms must navigate trade-offs among optimality, completeness, and real-time performance. Developing new path planning algorithms capable of efficiently identifying high-quality initial paths in large-scale, complex environments—while staying feasible on platforms with limited onboard computational resources—remains a significant challenge.

4. Trajectory Optimization

The trajectory optimization module is a fundamental component of motion planning as it refines the line segments paths produced by the path planning module — initial paths that lack time allocation — by optimizing them over time to yield a safe, smooth, and dynamically feasible trajectory. This optimized trajectory is executed directly by the UAV, so its efficiency and robustness are essential for maintaining safe autonomous flight. A robust trajectory optimization algorithm should prioritize both the quality of the resulting trajectory and the convergence speed, ensuring that the UAV can quickly adapt to unexpected obstacles during operation.

Trajectory optimization methods are generally categorized by their approach to constructing the objective function: hard-constrained and soft-constrained. In the hard-constrained approach, which is constrained for safety, control inputs and dynamics are represented as equations or inequalities, forcing the objective function to adhere strictly to these conditions. In contrast, the soft-constrained approach incorporates these constrained as penalty terms in the objective function, creating an unconstrained nonlinear problem where the optimization process inherently drives the trajectory to comply with the desired conditions. Figure 16 illustrates some of the representative work in both hard-constrained and soft-constrained approaches.

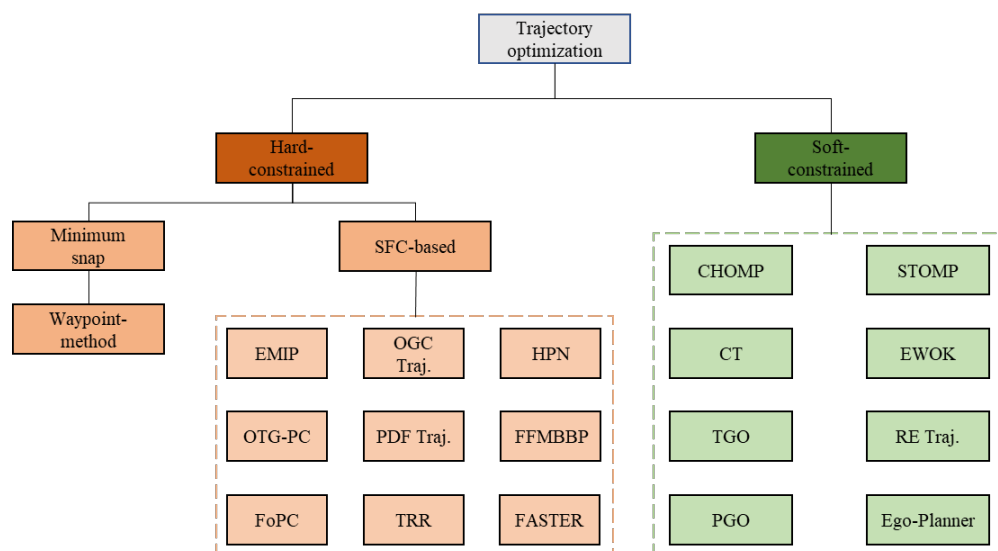


Figure 16. Classification of trajectory optimization algorithms.

4.1. Hard-Constrained Trajectory Optimization

The minimum snap approach [78] demonstrates that UAVs exhibit differential flatness, which implies that motion planning only requires consideration of the 3D position and yaw angle. This property significantly reduces the parameters required in planning, thereby simplifying the algorithm’s complexity.

Since the optimization target in minimum snap [78] is polynomial coefficients, the process can sometimes produce singular values, leading to numerical instability. To address this, ref. [79] proposed a closed-form solution for minimum snap, transforming the optimization variable from polynomial coefficients—lacking clear physical meaning—to derivatives of each order of the trajectory position, which have tangible physical significance. Converting the original constrained Quadratic Programming (QP) problem into an unconstrained QP problem, thereby enhancing the speed and stability of the solution.

Moreover, as the minimum snap [78] optimization process does not inherently account for surrounding obstacles, the generated trajectory may risk collisions, as illustrated in Figure 17. The gray and red areas indicate obstacles, while the black dashed line represents the segmented path obtained from the front-end search. The minimum snap-optimized trajectory (shown by the red curve) may intersect with obstacles, as illustrated in Figure 17a. To mitigate this issue, ref. [79] proposed a method that introduces additional waypoints (red dots in Figure 17b) to avoid obstacles. The optimization process is then constrained to pass through these waypoints, resulting in a smooth, collision-free trajectory, as depicted by the green curve in Figure 17b. In complex environments, however, frequent waypoint insertion may be necessary, requiring repeated executions of the optimization process to find feasible solutions. The added waypoints also increase the complexity of the QP, which can diminish the algorithm’s efficiency for real-time planning in intricate environments.

To tackle the high computational costs associated with frequent waypoint insertion, ref. [80] proposed a UAV trajectory optimization approach based on a Safe Flight Corridor (SFC). This method utilizes iterative region inflation by semidefinite programming (IRIS) [81] to model the environment’s collision-free areas as an SFC, and it consists of finite convex polyhedra. Mixed-integer second-order cone programming (MISOCP) is then applied to optimize this SFC. Additionally, the polynomial sum of squares (SOS) technique is used to express the linear constraints of the polynomial trajectories within the SFC, ensuring the trajectories remain collision-free.

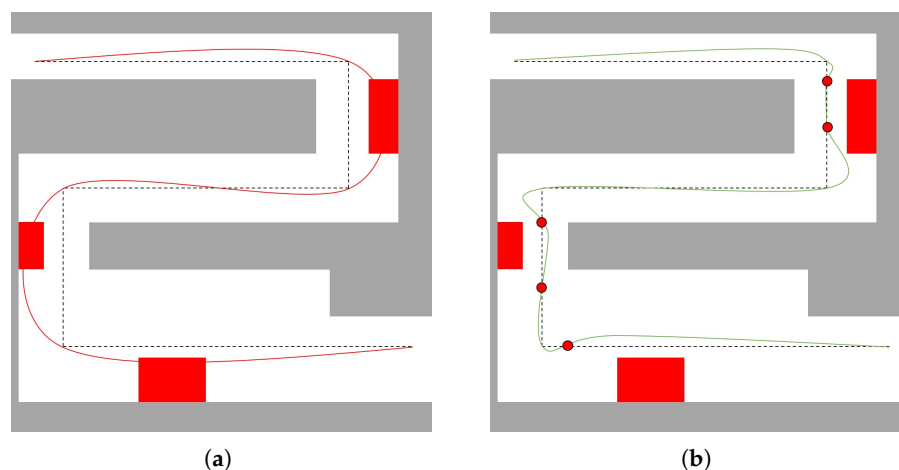


Figure 17. Schematic diagram of the trajectory generated by minimum snap. (a) Trajectory optimized by minimum snap without considering obstacle constraints, resulting in intersections with obstacles (red curve). (b) Additional waypoints (red dots) are strategically introduced to circumvent obstacles, ensuring the generated minimum snap trajectory (green curve) is collision-free.

Inspired by IRIS [81], ref. [82] proposed an algorithm for real-time SFC construction within OGMs [35]. This approach continuously updates the OGM using onboard sensors, and it utilizes A* [59] to quickly identify a collision-free geometric path from the start to the target point. The path is then sampled via ray tracing to determine the maximum collision-free interval, from which a polyhedral, collision-free corridor is constructed. Subsequently, a QP problem is formulated to optimize the trajectory within this SFC, producing a collision-free and smooth trajectory.

To further improve SFC construction efficiency, ref. [83] introduced an octree-based SFC construction method, which greatly accelerates convergence. This method leverages Octomap [36] for map representation, with A* [59] being employed to efficiently search for an initial path consisting of 3D grid cells. These cells are then expanded to form the SFC within the octree structure. The trajectory is represented using segmented polynomials from the minimum snap approach [78], and numerous inequality constraints from the SFC are used to formulate a QP, resulting in a safe, smooth, and dynamically feasible trajectory within the free space.

While these methods achieve fast SFC construction, their map representations require substantial computational resources and memory. To address this, ref. [34] proposed a method for constructing the SFC directly within a point cloud using a KD-tree [45] to efficiently organize and manage the point cloud data and perform rapid neighbor searches. Building on this structure, the authors applied RRG [73] to generate a series of spherical safe regions and used A* [59] to identify a path within these regions, resulting in a spherical SFC. The SFC is subsequently integrated into a QP to generate the trajectory.

To enhance both SFC construction efficiency and trajectory quality, ref. [84] introduced the regional inflation by line search (RILS) SFC construction and trajectory optimization framework, building on ref. [82]. Here, JPS [65] replaces A* [59] for front-end path planning, which significantly reduces computation time. A constrained QP problem was then formulated with linear inequality constraints from the SFC to generate a safe and smooth trajectory, as outlined in [82]. Figure 18 illustrates an example of the SFC construction when using the RILS method. The SFC is progressively built along the initial path: large spheres are first generated with diameters matching each path segment. These spheres are iteratively contracted to fit within collision-free areas, resulting in a sequence of maximal,

collision-free ellipsoids. Each ellipsoid is then expanded until it contacts an obstacle, where tangent surfaces form the final convex polyhedron, completing the SFC.

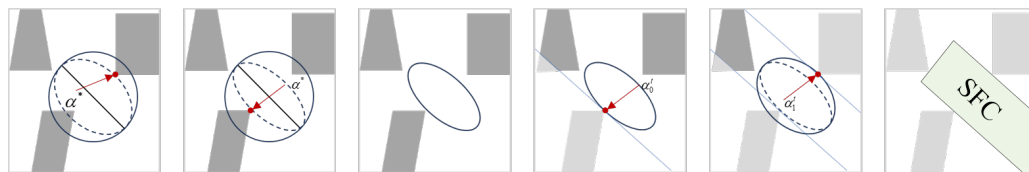


Figure 18. The process of constructing an SFC by RLS [84].

A common drawback of the above methods is their reliance on predefined time allocation along the trajectory. Although [79,84] have explored solutions to time allocation challenges, their initial estimates remain heuristic. Poor initial time estimates can hinder convergence to a globally optimal solution and often necessitate multiple iterations, leading to high computational costs. To address this, ref. [85] introduced a fast marching method to enable more effective time allocation, thereby enhancing trajectory quality. This approach begins by constructing a velocity field based on the ESDF [38], which is followed by applying the fast marching method to rapidly determine a path from the start to the target within the velocity field, naturally integrating time information. Subsequently, the distance data from the ESDF allow for checking each node's proximity to obstacles, facilitating the construction of a collision-free bounding box around each node to form the SFC. Finally, Bézier curves [86] are used to parameterize the trajectory, and, by solving a QP problem with comprehensive time allocation, a smooth, and dynamically feasible trajectory is achieved.

To further improve computational efficiency and numerical stability in trajectory optimization, ref. [87] built on the work of [34] by implementing an incremental path-finding module, enabling continuous Safe Flight Corridor (SFC) optimization during UAV flight. The trajectory is parameterized with segmented Bézier curves [86], facilitating the production of safe, dynamically feasible paths through a single convex optimization problem. Additionally, the transformation of the quadratically constrained quadratic program (QCQP) problem into a second-order conic program (SOCP) problem enhances both numerical stability and computational efficiency.

To address the limitations of solution space in the constructed SFC, which can lead to conservative trajectories, FASTER [88] was introduced to enable rapid UAV navigation in unknown environments. FASTER uses a hierarchical trajectory planning framework, where a long-horizon global planner guides a short-horizon local planner toward the target. The global planner employs JPS [65] to create a segmented linear trajectory from the start to the goal. For the local planner, FASTER simultaneously plans two optimal trajectories: one in a known safe region for safety assurance and another in an unknown region to maintain aggressiveness. Extensive testing in both simulated and real-world environments has shown that FASTER enables UAVs to reach speeds of up to 3.6 m/s in complex, unknown settings.

Ref. [89] introduced the TRR framework to achieve spatial and temporal optimization of UAV trajectories. This approach designs a parallel convex cluster inflation (PCCI) algorithm for SFC construction based on OGMs [35]. TRR starts with an unoccupied grid as a seed, growing layer by layer and checking the visibility of each new grid relative to the existing set, thereby preserving the convexity throughout the growth. The convex hull of these grids serves as an SFC approximation, maximizing the size of each convex polyhedron and expanding the solution space. Spatial and temporal optimization then occur alternately within the SFC. For spatial optimization, Bézier curves [86] are used to parameterize the trajectory, with control points adjusted to ensure the trajectory remains within the SFC for safety. For temporal optimization, time-optimal path parameterization [90] is applied.

In summary, hard-constrained trajectory optimization methods generate feasible trajectories by formulating a QP problem with constraints on safety, smoothness, and dynamic feasibility, which are solved via convex optimization techniques. However, these methods often require constructing an SFC to supply numerous linear or equality constraints and involve iterative QP solving to ensure trajectory safety, which considerably increases computational time. Furthermore, since SFC construction does not consider the distance between the trajectory and obstacles, the resulting paths may end up too close to obstacles, as illustrated in Figure 19.

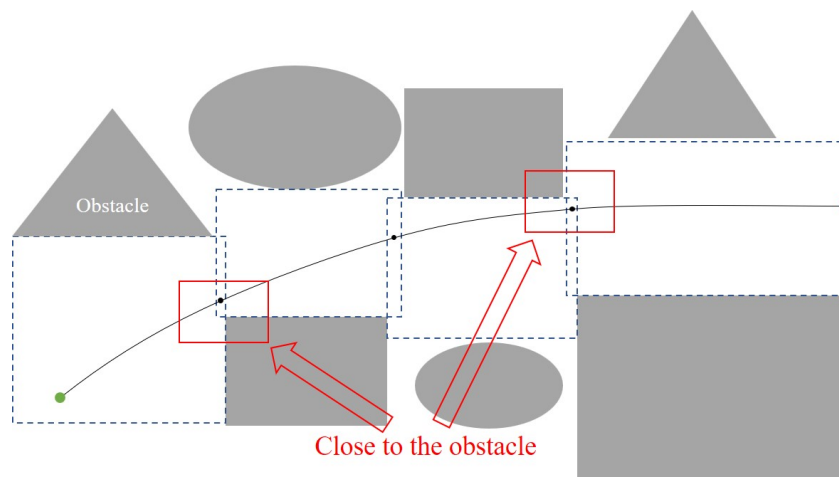


Figure 19. Hard-constrained methods tend to generate low clearance trajectory.

4.2. Soft-Constrained Trajectory Optimization

In contrast to the hard-constrained approach—where safety, smoothness, and dynamic feasibility are enforced through equation or inequality constraints within a QP that is solved with a dedicated solver—the soft-constrained approach incorporates these requirements as penalty terms within the objective function itself. By embedding these constraints as penalty terms, the method formulates an unconstrained nonlinear optimization problem that directly optimizes the objective function. This approach inherently steers the resulting trajectory to meet safety, smoothness, and dynamic feasibility conditions, as illustrated in Figure 20.

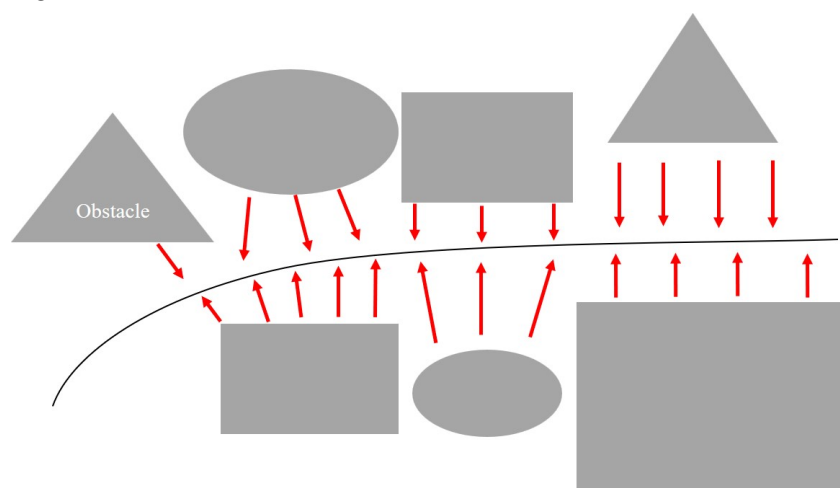


Figure 20. Soft-constrained methods use gradient descent to “push” trajectories away from obstacles.

CHOMP [51] pioneered the use of soft-constrained trajectory optimization by formulating an objective function where safety and smoothness appear as penalty terms. The method discretizes the trajectory into waypoints at fixed time intervals and applies

gradient descent to minimize the objective function, producing a safe and smooth path. CHOMP also makes innovative use of the detailed gradient information available from the ESDF [38], which significantly accelerates collision detection.

STOMP [91] is a stochastic trajectory optimization method inspired by CHOMP [51]. Unlike CHOMP, STOMP employs a sampling-based method to generate a set of candidate trajectories and evaluates them using importance sampling. This approach allows STOMP to handle non-convex constraints and objective functions, achieving computational efficiency in some cases.

While CHOMP and STOMP can produce kinematically feasible trajectories, they do not account for dynamic constraints. Addressing this, CT [92] represents the trajectory as segment polynomials, as proposed in the minimum snap method [78]. Using OGM [35] and ESDF [38] as map representation, the approach begins with Informed-RRT* [76] to find a collision-free line segment path, and it then refines the trajectory to meet dynamic constraints using the method in [93]. Compared to CHOMP, CT has a higher success rate and greater efficiency in complex environments without requiring trajectory discretization or extensive parameter tuning.

EWOK [49] introduces B-splines [94] for trajectory representation. Unlike polynomial-based approaches, B-splines offer higher continuity with fewer degrees of freedom and do not need explicit boundary constraints for optimization. B-splines also provide local support, where each control point influences only a specific trajectory segment, thereby reducing the number of optimization variables and constraints. EWOK further optimizes computational and memory efficiency by implementing a 3D circular buffer for storing a local map around the UAV, achieving substantial improvements in computation time compared to CT [92] while maintaining comparable success rates and path lengths.

To further enhance the success rate of trajectory optimization, TGO [95] was introduced. TGO separates the trajectory generation process into two main stages: collision-free path generation and trajectory optimization. First, it uses the Informed-RRT* [76] method to establish an initial collision-free geometric path, which is then parameterized using segmented polynomials. Next, an objective function incorporating smoothness, collision avoidance, and dynamic feasibility penalty terms is formulated and optimized to determine the optimal polynomial coefficients. This optimization proceeds in two steps: initially optimizing the collision term, followed by time redistribution to ensure dynamic feasibility. TGO leverages KD-tree [45] for nearest neighbor searches along the trajectory, enabling it to adjust the trajectory away from obstacles for enhance safety. When trajectory adjustments are made, time reallocation is used to maintain dynamic feasibility. The method demonstrated a 100% success rate at an obstacle density of 1.2 obstacles/m², significantly outperforming CHOMP [51] and CT [92], both of which showed reduced success rates as obstacle density increased.

To further enhance the quality of trajectory, ref. [96] introduced RE-Traj. This method initially searches for a kinodynamic trajectory using Hybrid A* [66]. For back-end optimization, the trajectory is parameterized with a uniform B-spline, which is followed by time reallocation to yield a non-uniform B-spline trajectory that satisfies dynamic feasibility and increases aggressiveness. In comparisons with TGO [95] for motion planning, RE-Traj demonstrated substantial improvements in computation time and trajectory length, producing smoother trajectories in less time.

The methods discussed above can effectively generate safe trajectories in complex environments; however, gradient-based approaches are prone to becoming trapped in local minima, as shown in Figure 21. To address this issue, ref. [97] was proposed, employing a roadmap construction technique similar to that in [98] to form a compact roadmap and search for multiple topological trajectories within it. PGO also introduces uniform

visibility deformation (UVD) to swiftly reduce redundancy among topologically consistent trajectories. If a trajectory encounters a local minimum, the algorithm initiates a search for alternative topological paths, which is followed by the optimization of each path. The optimization process consists of two steps: first, generating a collision-free geometric path, and then refining the trajectory using B-splines and ESDF [38] to achieve a safe and smooth final trajectory. Comparative experiments with EWOK [49] and TRR [89] demonstrate significant gains in both success rates and trajectory smoothness, with only a slight increase in computational time.

However, constructing and maintaining an ESDF requires substantial memory and computational resources, and only a small portion of the obstacle gradient information provided by the ESDF is utilized during trajectory optimization, leading to further resource waste. To address this issue, ref. [99] introduced Ego-planner, a soft-constrained framework for trajectory optimization that operates without relying on an ESDF. This method first uses A* to quickly find a piecewise linear path that bypasses obstacles when the initial trajectory intersects with them, and it then generates a set of thrusts based on this path to pull the trajectory away from the obstacles. Finally, an anisotropic fitting method is applied for time reallocation, preventing trajectory overshoot. Experimental results show that this approach achieves a convergence rate over 80% faster than EWOK [49] and RE-Traj [96].

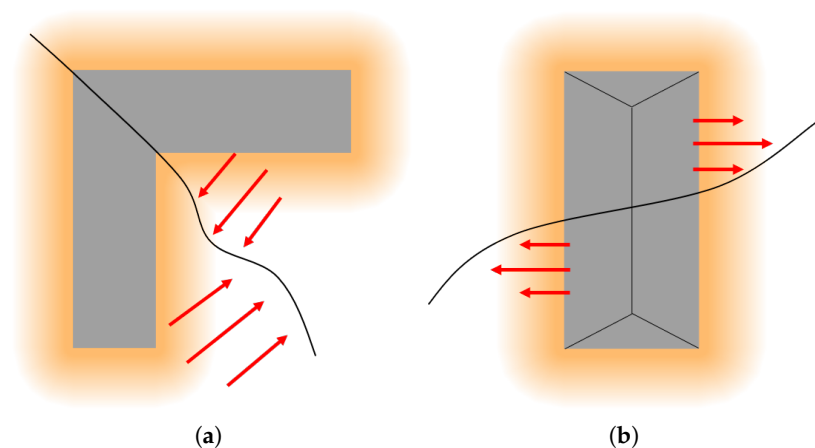


Figure 21. Typical examples of optimization failure, which occurs since it crosses the “valley” (a) and the “ridge” (b) of the ESDF.

The soft-constrained approach formulates a nonlinear optimization problem by integrating constraints such as safety, smoothness, and dynamic feasibility as penalty terms, thereby solving for a feasible trajectory using gradient descent. However, this approach has notable drawbacks. First, it typically requires a suitable potential collision cost function, like an ESDF [38], to evaluate obstacle gradients effectively. Constructing an ESDF is resource-intensive, consuming substantial memory and computational time, which reduces trajectory optimization efficiency. Second, the method is vulnerable to local minima, potentially leading to optimization failures.

4.3. Discussion

To provide a clearer comparison of the performance across different types of trajectory optimization algorithms, Table 3 provides an overview of the contributions and limitations of various classical trajectory optimization algorithms. Hard-constrained methods formulate a QP problem and utilize generic convex solvers to generate trajectories. Although these methods often produce high-quality trajectories, they are computationally demanding, sensitive to noise, and may yield conservative paths that do not fully leverage available space. In contrast, soft-constrained methods apply gradient descent to minimize

a cost function, achieving computational efficiency but without guarantees of optimality, and they are susceptible to local minima, which can result in optimization failure.

Table 3. Several classical path-finding algorithms and their advantages and limitations.

Method	Ref.	Contributions	Limitations
Hard-constrained	[78]	- Introduces differential flatness; - Represents full quadrotor state using 3D position and yaw angle, reducing state complexity.	- May generate unsafe trajectory; - Initial time allocation for each segment requires heuristic estimation.
	[79]	- Closed-form solution of [78], which improves numerical stability; - Inserting key waypoints helps avoid collisions.	- Increases computational cost and complexity; - Time allocation needs to be calculated iteratively.
	[80]	- Uses IRIS to partition space into convex regions, creating an SFC; - SOS function guarantees trajectory remains within SFC.	- Computationally intensive; - Reducing suitability for real-time applications.
	[82]	- Employs A* for path planning and inflates path to obtain SFC; - Defines a hover strategy to ensure flight safety.	- Limited by sensor FoV; - Conservative trajectory.
	[83]	Fast SFC construction based on Octomap.	Requires iterative optimization to avoid collisions.
	[34]	- Constructs SFC in point cloud; - Solve QCQP to obtain trajectories within SFC.	Constraints may result in excessive deviation from the UAV's goal.
	[84]	- Introduces RILS to construct SFC; - Utilizes RHP strategy to ensure safety.	SFC construction can be overly conservative, reducing solution space.
	[85]	A fast marching-based method to achieve better time allocation.	The final trajectories do not strictly follow the heuristics after optimization.
Soft-constrained	[87]	- Proposes an incremental SFC construction method; - Converts QCQP to SOCP, improving numerical stability and computational efficiency; - Uses segmented Bézier curves for trajectory representation.	SFC construction can be affected by noise, leading to conservative solution spaces.
	[88]	- Maintains two trajectories to enhance flight aggressiveness; - Introduces a heuristic approach to improve directionality in optimization.	Requires calculation of multiple trajectories simultaneously.
	[89]	Alternates between spatial and temporal trajectory optimization.	High computational complexity for SFC construction and temporal allocation.
	[51]	- Constructs an unconstrained objective function with gradient descent, facilitating trajectory optimization; - Utilizes ESDF for efficient collision cost evaluation.	Prone to becoming stuck in local minima.
	[91]	Uses a stochastic optimization approach to avoid local minima.	Results in disconnected paths that may not satisfy dynamic constraints.
	[92]	Generates dynamically feasible piecewise trajectories for complex environments.	Requires predetermined trajectory segments, which limits applicability.
	[49]	- Uses a 3D critical buffer to maintain local map, reducing memory load. - Expresses trajectories with B-splines, ensuring safety and dynamic feasibility.	Not applicable to large-scale complex scenarios.
	[95]	Two-stage optimization strategy improves success rate.	Homotopy classes of initial geometric paths may lack optimal trajectory solutions.
	[95]	Two-stage optimization strategy improves success rate.	Homotopy classes of initial geometric paths may lack optimal trajectory solutions.
	[96]	- Enhancing optimization success rate through using a front-end dynamic path planning; - Uses non-uniform B-splines for dynamic feasibility.	Dynamical space search may miss some solutions.
	[97]	Topological path guided approach minimizes risk of local minima.	The process of generating topological trajectories is time-consuming.
	[99]	Does not rely on ESDF, saving memory and computational costs.	- Low trajectory quality; - Unified time reallocation tends to obtain conservative trajectories.

In summary, both hard-constrained and soft-constrained methods offer distinct advantages and challenges. The ongoing goal for researchers is to develop methods that can consistently produce safe, smooth, and dynamically feasible high-quality trajectories within limited computation time to make them suitable for real-time applications.

5. Related Framework Review and Specific Case Studies

5.1. Related Framework Review

Building on the earlier discussion of the three core modules of UAV motion planning—map representation, path planning, and trajectory optimization—this section examines their integration into state-of-the-art frameworks. To enable safe autonomous

navigation in unknown and complex environments, UAV motion planning algorithms must effectively address these three modules. Table 4 provides an overview of UAV motion planning frameworks validated in real-world scenarios over the last four years. These algorithms are categorized based on the core modules, offering a structured perspective on recent advancements in algorithmic design and practical deployment.

Table 4. Real-world validated UAV motion planning framework during 2021–2024.

Ref.	Map Representation	Path Planning	Trajectory Optimization	Contributions	Limitations
[100]	PCM	Searching-based	Soft-constrained	Enables efficient detection of both static and dynamic obstacles.	Perception Range Limitations.
[101]	OGM and ESDF	Sampling-based	Soft-constrained	- Enhanced visibility and safe reaction distances for unknown obstacles; - Enables proactive exploration of unknown environments to gather essential information.	Introducing safety and view constraints increases optimization complexity.
[102]	PCM	Sampling-based	Hard-constrained	- Efficiently maintains historical sensor information and supports reliable queries; - Utilizes a Forward Spanning Tree to extract the free space skeleton and generate a SFC.	Additional waypoints may be needed in some cases.
[103]	OGM and ESDF	Searching-based	Soft-constrained	- Leverages an improved A* algorithm to perceive the surrounding environment; - Selects appropriate initial path modes for navigating narrow spaces based on guided path.	Guide path generation and update in complex dynamic environments may experience delays.
[104]	OGM	Sampling-based	Hard-constrained	Introduced an environment-guided sampling to enhance the Kinodynamic-RRT*.	Sensitive to the initial trajectory.
[105]	OGM and ESDF	Searching-based	Soft-constrained	- Prioritized obstacles based on environmental risk levels and motion trends; - Developed a multi-layer model predictive contouring control to generate trajectories.	Does not account for environmental factors, such as wind and sensor inaccuracies.
[106]	OGM and ESDF	Searching-based	Soft-constrained	- Proposed a body airflow disturbance model and validated on real quadrotors; - Considering the control error bound utilizing the FRSSs computed from the estimated disturbances.	Disturbance estimation and adaptation for autonomous outdoor flight can be challenging.
[107]	OGM	Searching-based	Soft-constrained	- Collision risk is incorporated as a penalty term in a nonlinear optimization framework to enable mutual collision avoidance between UAVs; - Introduced a topological trajectory generation method to avoid local minima; - Utilized proxy detection from depth images to correct relative position drift.	Unreliable communication networks or limited bandwidth may hinder UAV operation.
[108]	OGM and ESDF	Searching-based	Soft-constrained	Used a dynamic environment perception method that accounts for occlusions.	Model may be too simplified to accurately model more complex obstacles.
[109]	OGM and ESDF	Searching-based	Hard-constrained	- Decoupled spatial and temporal variables to enable trajectory optimization. - Utilized convex optimization for spatial variables and gradient descent for time variables.	The computational complexity is relatively high.
[110]	OGM	Sampling-based	Hard-constrained	Utilized the multi-resolution state lattices, enhances computational efficiency.	Potential disturbances in disaster have not been considered.
[111]	OGM	Sampling-based	Hard-constrained	- Introduced MINCO trajectory representation, to enable efficient spatiotemporal joint optimization. - Enables elimination of geometric constraints without introducing additional local minima.	Prone to local minima, leading to suboptimal trajectories.
[112]	OGM	Sampling-based	Hard-constrained	- Designed a palm-sized micro-swarm platform, integrating perception, localization, and control functionalities. - Developed a spatiotemporal trajectory planner to simultaneously optimize trajectory shape and time allocation.	Accumulated SLAM drift could lead to UAV collisions.

Table 4. Cont.

Ref.	Map Representation	Path Planning	Trajectory Optimization	Contributions	Limitations
[113]	OGM and ESDF	Searching-based	Hard-constrained	- Proposed a differentiable metric to quantify the similarity between 3D configurations. - Established a unified optimization framework to jointly optimize configuration similarity, collision avoidance, and dynamic feasibility.	The computational complexity is relatively high.
[114]	OGM	Searching-based	Hard-constrained	- Proposed a dynamic free-space path graph for safe quadrotor motion planning. - Introduced an efficient method to incrementally update a polyhedral map in dynamic environments.	Does not consider noise and completeness of SLAM data.
[115]	OGM	Searching-based	Hard-constrained	- Employed a linear safe corridor to formulate a distributed optimization problem. - Proposed a priority-based goal-planning method to prevent deadlock.	Assumes initial trajectory is collision-free.
[116]	Octomap	Sampling-based	Hard-constrained	- Introduced a sampling-based SFC generation method, maintaining a larger corridor volume. - Proposed a RHC scheme to ensure that the corridor always includes sufficient space.	Not always guaranteed to obtain obstacle-free SFC.
[117]	OGM and ESDF	Searching-based	Hard-constrained & Soft-constrained	- Adopted a combined soft and hard constrained method to obtain high-quality trajectories. - Designed an inert replanning strategy to maintain trajectory quality while reducing the frequency of replanning.	Switching between optimization methods adds complexity.
[55]	OGM and SM	Searching-based	Hard-constrained	- Adopted a multi-level planning and mapping framework to construct semantic maps of areas of interest. - Used semantic SLAM to minimize odometry drift while maintaining optimality in the planner and stability in the controller.	Conservative navigation in cluttered environments limits speed.
[118]	OGM and ESDF	Searching-based	Hard-constrained	- Used real-time spatiotemporal planning method based on differential flatness. - Designed NMPC controller featuring a hierarchical disturbance compensation strategy.	Difficult to align payload with center of mass.
[119]	OGM and ESDF	Searching-based	Hard-constrained	- Used a multi-UAV tether-aware homotopy representation. - Developed a complete tether-aware planning framework.	Limited vertical navigation capability.
[120]	OGM and ESDF	Searching-based	Hard-constrained	Velocity constraints are transformed into step-size constraints between adjacent B-spline control points.	End velocity and acceleration must be zero.
[121]	OGM	Searching-based	Hard-constrained	Proposed a method to construct SFC for UAVs using geometric space expansion and segmentation techniques.	Some safe flight regions are overlooked.
[122]	OGM	Sampling-based	Hard-constrained	- Used a differentiable graph-based similarity metric to quantify formation similarity. - Used a distributed trajectory optimization framework. - Adopted an adaptive formation reconfiguration method.	Weak dynamic environment adaptability.
[123]	OGM and ESDF	Searching-based	Hard-constrained	Developed an enhanced A* method considering safety and search efficiency.	Spherical safe regions lead to overly conservative behavior.
[124]	OGM	Searching-based	Soft-constrained	- Introduced a polyhedral-based environmental modeling to improve environmental feature extraction. - Presented a path search method based on local topological maps.	Only validated in static environments, limiting applicability.
[125]	OGM and ESDF	Searching-based	Soft-constrained	- Proposed a visibility-based trajectory optimization method to handle occlusion. - Developed a view-based metric to prevent target occlusion by obstacles effectively. - Incorporated environmental complexity to adjust the UAV's relative distance from targets.	Relies solely on distance to obstacles for occlusion metrics.

Table 4. Cont.

Ref.	Map Representation	Path Planning	Trajectory Optimization	Contributions	Limitations
[126]	PCM	Searching-based	Hard-constrained	- Proposed a novel method for dynamic object speed measurement based on point cloud sequences. - Considered sensor FOV and dynamic object velocity model uncertainty. - Introduced a yaw planning algorithm to improve object tracking and flight safety.	Simplistic constant velocity model for dynamic obstacle prediction.
[127]	OGM and ESDF	Searching-based	Soft-constrained	Developed a tight collision probability bound supporting safe motion planning.	Multiple iterations are required, increasing computational load.
[128]	OGM	Sampling-based	Soft-constrained	- Proposed a particle-based risk assessment method capable of handling static and dynamic obstacles of arbitrary shapes. - Created an efficient risk-aware sampling-based local trajectory planner.	Particle prediction involves uncertainty, affecting robustness.
[129]	OGM	Searching-based	Hard-constrained	Developed an integrated planning and control method for quadrotor navigation in obstacle avoidance situations.	Only safety metrics are considered, ignoring energy and flight time.
[130]	OGM and ESDF	Searching-based	Soft-constrained	Validated the feasibility of autonomous navigation for under-canopy surveying using an autonomous UAV laser scanning system.	Requires further validation in multi-layer forest environments.
[131]	OGM	Searching-based	Soft-constrained	Used lightweight target detection algorithms and KF to detect, track, and predict the movement of dynamic obstacles.	KF may not adequately predict more complex obstacle behaviors.
[132]	PCM and OGM	Sampling-based	Hard-constrained	Proposed a distributed and asynchronous navigation framework leveraging CPU and GPU for improved real-time performance.	May be constrained by communication range and susceptibility to signal interference.
[133]	OGM and ESDF	Searching-based	Soft-constrained	Introduced a yaw-safe corridor for enhanced safety, along with a Fisher Information Field to assess perception quality.	Yaw safety treated as a soft constraint, risking violations.
[134]	OGM and ESDF	Searching-based	Soft-constrained	Incorporated control barrier functions and disturbance observers into MPCC constraints to manage dynamic obstacles and external disturbances.	Applicability in more complex environments requires further validation.
[135]	OGM and ESDF	Searching-based	Soft-constrained	- Selected frontiers based on exploratory frontier points and decision functions, reducing redundant paths. - Enhanced environmental adaptability by actively sensing obstacle size and switching planning strategies.	Adaptability to highly dynamic environments may be limited.
[136]	Octomap	Searching-based	Soft-constrained	Generated inspection flight paths based on automatically extracted blade parameters.	Insufficiently smooth transitions may affect stability.

In 2021, Kong et al. [100] designed a compact autonomous UAV with LiDAR as the detection sensor, and they also used Hybrid A* [66] for trajectory search to automatically avoid small obstacles. Zhou et al. introduced RAPTOR [101], which adaptively adjusts the distance between trajectories and obstacles while planning yaw angles based on real-time risk assessments from the surrounding environment. This approach enhances UAV safety during high-speed flights by proactively managing proximity to obstacles. Ji et al. [102] developed a map-free UAV motion planning algorithm utilizing PicoMap, a novel data structure designed for efficient maintenance and rapid querying of sensor historical data. The algorithm features an enhanced sampling method for generating a SFC, and it introduces an intelligent waypoint selection strategy to optimally distribute waypoints within the SFC. This integrated approach facilitates the generation of a safe, dynamically feasible trajectory that adapts effectively to environmental constraints. Zhao et al. [103] introduced an improved A* to search a piecewise straight linear and to detect narrow passages, and if a narrow passage was detected, Hybrid A* [66] is subsequently used. When used, it is guided by the initial path to find a trajectory that satisfies dynamic constraints, thereby improving optimization success rates in complex environments.

Wang et al. proposed TGK-Planner [104], which uses a bootstrap sampling-based Kinodynamic-RRT* [77] for the rapid searching of a safe and dynamically feasible initial

trajectory at the front end. The planner then formulates an objective function involving smoothness, homotopy, and continuity, and it is solved using a closed-form approach. Quan et al. proposed EVA-Planner [105], an environmental adaptive motion planning framework that dynamically adjusts flight aggressiveness based on environmental complexity and the UAV's current state. Wang et al. [106] considered UAV self-induced airflow disturbances by modeling the disturbance using the acceleration variance from hovering states, calculating the reachable set based on the disturbance model and utilizing Hybrid A* [66] with B-splines [94] for dynamic path planning and trajectory optimization to achieve autonomous flight.

Zhou et al. proposed EGO-Swarm [107], which extends the EGO-Planner [99] by integrating topological planning methods for enabling collaborative multi-UAV flight. Wang et al. [108] developed an occlusion-aware dynamic perception algorithm that classifies obstacles into dynamic and static types, represents dynamic objects with moving ellipses, integrates static objects into an occupancy grid map, and designs a modified kinodynamic path planning and gradient-based optimization algorithm for dynamic obstacle avoidance. Sun et al. [109] introduced a bilevel optimization method where the lower level constructs the optimal state by combining the SFC to solve the QP problem. The upper level then performs direct time-domain optimization using the analytic sensitivity derived from the underlying constrained QP solution. Schleich et al. [110] employed a multi-resolution state lattice approach, where spatial resolution decreases with increasing distance from the current MAV position, effectively reducing computational costs.

In 2022, Wang et al. proposed GCOPTER [111], which utilizes a novel trajectory representation named MINCO (minimum control) with sparse parameters. This representation enables a joint spatial-temporal optimization approach that supports any user-defined general input. Zhou et al. [112] introduced a framework for simultaneous obstacle avoidance and formation flight planning for UAV swarms in complex environments. The framework employs a differential graph theory-based cost function to quantify the similarity distance between 3D formations. To maximize formation similarity while maintaining obstacle avoidance capabilities, the framework optimizes a cost objective function that considers dynamic feasibility, safety, and formation similarity for swarm formation flight in dense environments. Quan et al. [113] developed a miniature fully autonomous UAV and proposed a multi-agent collaborative motion planning framework. The proposed approach utilizes the method detailed in [111] representing trajectory, enabling direct spatial-temporal optimization of trajectories to produce high-quality paths within milliseconds in environments with limited computational resources.

Park et al. [115] proposed a linear safety corridor (LSC) approach for multi-UAV online motion planning, addressing the local optima issues commonly seen in optimization-based methods. Ren et al. [116] introduced receding horizon corridors (RHCs) to ensure sufficient safe space for re-planning, thereby improving both re-planning success rates and convergence during high-speed UAV flight. Yan et al. [117] combined soft and hard constraint methods by initially generating a high-quality trajectory using GCOPTER [111], which was followed using EGO-planner [99] for rapid local obstacle avoidance, thus ensuring safe and efficient autonomous flight. Liu et al. [55] achieved robust and accurate positioning using semantic SLAM with drift compensation mechanisms. A multi-level planning and mapping framework enhances computational and storage efficiency, enabling autonomous flight and real-time semantic mapping in large, unstructured environments.

In 2023, Li et al. [118] proposed a UAV load transport planning and control framework that integrates real-time planning for smooth trajectories, considering system dynamics with a hierarchical adaptive control mechanism to counteract disturbances and model inaccuracies. Cao et al. [119] presented a decentralized non-entangled trajectory generation

approach for multi-UAV systems. This method employs a novel tether-aware homotopy representation to evaluate the entanglement risks and required cable length, combining kinodynamic path planning and soft constraint optimization to produce high-quality trajectories. Wang et al. [120] utilized B-splines to transform velocity constraints into adjacent control point constraints, applying a MINVO basis [137] for trajectory representation and solving a QCQP problem to achieve smooth, adaptive-speed, safe, and dynamically feasible trajectories within the SFC. Ma et al. [121] proposed a polyhedral SFC construction technique using geometric expansion and slicing to improve efficiency and expand the solution space, enhancing UAV navigation robustness in complex, low-altitude environments.

Quan et al. [122] introduced a UAV formation system featuring a differentiable graph-based similarity metric for formation control. The system utilizes distributed optimization to consider formation similarity, safety, and dynamic feasibility, adapting parameters and task assignments for reliable, large-scale outdoor formation flying. Liu et al. [123] enhanced the A* path planning method to balance safety and efficiency in generating motion primitives, followed by a two-stage optimization framework that improves spatial position and trajectory safety. Gao et al. [124] used OGMs [35] to model obstacles as convex and concave polyhedra, providing comprehensive obstacle information for motion planning, thereby generating high-quality, safe, and smooth local trajectories.

Wang et al. [125] introduced a safety–visibility-guided optimization approach that adjusts UAV distance and yaw angle relative to obstacles, which reduces occlusion effects and avoiding collisions. Chen et al. [126] developed a method to estimate the dynamic object velocity from point cloud sequences, overcoming self-occlusion issues. This approach incorporates trajectory optimization for both dynamic and static objects, ensuring feasible flight paths, and it includes a yaw planning algorithm to enhance flight safety. Liu et al. [127] presented a compact collision probability bound for reliable motion planning in dynamic environments with external disturbances. Chen et al. [128] proposed a particle-based risk assessment method for the real-time perception of static and dynamic obstacles without target detection, as well as a risk-aware sampling-based local trajectory planner with a two-stage trajectory fusion strategy.

In 2024, Liu et al. [129] introduced an integrated planning and control trajectory generation framework, combining a front-end path planning module with a back-end MPC module for low-latency optimization, thus demonstrating superior performance in sudden obstacle avoidance and disturbance rejection. Liang et al. [130] deployed a self-developed autonomous UAV laser scanning system that incorporates hierarchical path planning to achieve high-precision data acquisition in under-canopy environments. Zhong et al. [131] utilized a lightweight object detection algorithm coupled with Kalman filtering to track and predict dynamic obstacles, integrating a B-spline [94] optimization method for safe and agile navigation in dynamic environments. Zhang et al. [132] employed a spherical coordinate-based point cloud mapping strategy accelerated by GPU, along with a hierarchical motion planning approach that uses OGM [35], for initial path generation and MPPI for back-end optimization, enabling multi-UAV navigation in unknown environments.

Sun et al. [133] introduced a yaw-safe flight corridor to ensure safety, and this was combined with a Fisher Information Field (FIF) to evaluate perception quality, where a coarse-to-fine search strategy is employed for high-precision localization. Qiu et al. [134] integrated control barrier functions with disturbance observer techniques to construct a model predictive contouring control (MPCC) model, facilitating aggressive, safe, and robust navigation in dynamic, disturbance-prone settings. Shi et al. [135] employed frontier point selection to guide the local planner, effectively reducing redundant paths and enhancing planning success rates while dynamically adjusting obstacle detection based on their size for improved environmental adaptability. Yang et al. [136] presented BladeView, a method

that constructs SFC and employs B-splines to fit the center line of the SFC, enabling fully automated inspection of wind turbine blades.

5.2. Specific Case Studies

To further demonstrate the practical effectiveness of the three key technologies discussed in this paper within UAV motion planning, we reviewed several widely recognized and high-performing case studies.

RAPTOR [101] is a robust and perception-aware motion planning framework tailored for high-speed UAV navigation in unknown environments. In terms of map representation, ref. [101] employs a volumetric mapping system based on OGMs [35], which is complemented by an ESDF [38] for gradient-based trajectory optimization and visibility evaluation. This enables real-time mapping and ensures precise collision checking. Regarding path planning, ref. [101] integrates a topological searching-based path planning to extract multiple topologically distinct guiding paths, enriching the exploration of the solution space and avoids local minima. These paths guide PGO, ensuring feasible and smooth trajectories, even in complex environments. For trajectory optimization, RAPTOR [101] introduces a two-phase soft-constrained optimization approach: an initial “warm-up” trajectory generation guided by geometric paths, and a subsequent risk-aware trajectory refinement. The refinement incorporates perception awareness by enforcing visibility toward unknown regions and optimizing yaw angles to maximize information gain. This ensures the UAV can actively observe hidden obstacles and react safely. The system is extensively validated through simulations and real-world experiments, showcasing its robustness and effectiveness for fast, safe flight in challenging scenarios.

Ref. [112] presents a comprehensive solution for multi-UAV autonomous navigation in complex and unknown environments. For map representation, they employed a probability-based occupancy grid [35] to capture environmental obstacles, which was augmented by onboard visual-inertial odometry (VIO) and ultra-wideband (UWB) range measurements to correct drift. This enables robust localization and obstacle detection, even in dense forests and dynamic scenarios. At the path-planning level, the work introduced a decentralized spatio-temporal joint optimization strategy, allowing the UAV swarm to avoid collisions both with one another and with environmental obstacles while still maintaining coordinated flight. This strategy proves effective for navigating through narrow corridors and highly cluttered obstacle distributions. Regarding trajectory optimization, the approach leverages the hard-constrained optimization method for the differentially flat systems proposed in [111], whereby spatial and temporal parameters are decoupled to reduce computational overhead. Multiple penalty terms for smoothness, flight time, collision avoidance, and dynamic feasibility are incorporated into a unified optimization framework, facilitating the real-time synthesis of high-quality motion trajectories. Extensive real-world experiments and simulations verify that, even under constrained hardware resources, this approach consistently achieves efficient swarm operations in challenging environments.

Ref. [138] introduced FASTER, a motion planning framework for UAVs that balances high-speed navigation with safety in unknown environments. For map representation, FASTER employs a body-centered sliding OGM [35], which ensures accurate and real-time environmental awareness while reducing the impact of estimation drift. In path planning, ref. [138] utilized the searching-based two-stage method. The global planner uses JPS [65] to compute a piecewise linear path through free-known spaces. Locally, FASTER [138] introduces a two-trajectory planning strategy that simultaneously generates (1) a “whole trajectory” that extends into unknown spaces to maximize speed and planning horizon, and (2) a “safe trajectory” that is strictly confined to free-known spaces to guarantee safety. The SFC, constructed through convex decomposition of the global path, provides collision-free

constraints for both trajectories. In trajectory optimization, FASTER adopts a Mixed-Integer Quadratic Programming (MIQP) formulation with hard constraints. These constraints ensure dynamic feasibility by enforcing strict limits on velocity, acceleration, and jerk while guaranteeing the trajectory remains entirely within the SFC. Extensive simulations and real-world experiments showcase the framework's ability to achieve speeds up to 7.8 m/s, outperforming prior methods in both speed and safety.

Ref. [139] presents SUPER, a safety-assured high-speed motion planning system designed for UAVs operating in unknown environments. In terms of map representation, SUPER utilizes a lightweight spatiotemporal sliding PCM [34] that is directly derived from LiDAR measurements. This approach eliminates the computational overhead of traditional OGMs [35] while maintaining centimeter-level accuracy, thus effectively preserving critical environmental details such as thin obstacles. For path planning, SUPER utilizes a searching-based two-trajectory planning strategy like [138]: the exploratory trajectory is generated both in free and unknown conditions to achieve aggressive flight; and the backup trajectory, confined strictly to free spaces, ensures safety and exhibits a more conservative planning paradigm. In trajectory optimization, SUPER employs a multi-stage polynomial parametrization framework that incorporates both soft and hard constraints. Soft constraints aim to minimize higher-order derivatives (e.g., jerk) to enhance trajectory smoothness and allocate time efficiently for spatio-temporal optimization. Hard constraints ensure dynamic feasibility by adhering to velocity and acceleration limits, as well as collision avoidance requirements within the planned corridors. By combining a two-trajectory planning strategy with efficient trajectory optimization, SUPER achieves a robust balance between safety and flight speed, demonstrating superior navigation performance in complex and cluttered environments.

6. Discussion and Future Directions

Motion planning is one of the critical technologies for achieving UAV autonomous navigation. Efficient motion planning methods enable UAVs to execute agile autonomous flight in highly challenging environments to accomplish specific tasks. This paper discusses each essential module in UAV motion planning, including map representation, path planning, and trajectory optimization, and it also provides an in-depth analysis of numerous representative works.

In the map representation section, this paper reviews several mainstream methods and evaluates their feasibility and limitations from perspectives such as memory burden, time complexity, and adaptability to different environments. Considering memory constraints and obstacle avoidance requirements, the combination of OGM [35] and ESDF [38] representations is suitable for most tasks, as shown in [96,101,140]. However, for large-scale motion planning tasks, PCM [34] is often preferred due to its superior memory efficiency, as shown in [87]. Conversely, when it comes to executing motion planning tasks that demand higher efficiency and accuracy, such as indoor search operations, SM [39] is typically a favorable choice [55]. Based on related research findings, it is clear that an effective map representation is still lacking, i.e., one that can minimize memory and computational complexity while accurately providing a potential cost function to estimate collision risk, thereby enhancing the safety and real-time performance of UAV motion planning.

In the path planning section, both searching-based and sampling-based approaches are shown to have their respective strengths. Searching-based methods, such as JPS [65] and A* [59], excel in structured, sparse environments where obstacles are regularly distributed, such as maze-like indoor maps [88,99]. These methods guarantee completeness and often yield near-optimal geometric paths, making them ideal for tasks requiring global optimality or environments with low clutter. However, they can struggle in complex, dense, or high-

dimensional environments due to computational overhead. In contrast, sampling-based methods, such as RRT* [73] and PRM [68], are more suited for unstructured or highly cluttered environments, such as forests or urban scenarios [104]. These methods efficiently handle high-dimensional state spaces and generate feasible trajectories quickly, making them preferable for real-time applications or scenarios with irregular obstacle distributions.

In the trajectory optimization section, both hard-constrained and soft-constrained methods are capable of generating high-quality trajectories, each excelling in different application scenarios depending on environmental complexity and task requirements. Hard-constrained methods, such as GCOPTER [111], are particularly suited for tasks demanding high trajectory accuracy, strict adherence to dynamic and input constraints, and effective time allocation. These characteristics make them ideal for structured or sparse environments (e.g., outdoor open scenarios) and missions that require long-range, high-speed maneuvers [113]. However, in highly cluttered environments, their performance may be hindered by increased computational complexity, reducing their success rate. Conversely, soft-constrained methods excel in dense, unstructured environments (e.g., indoor maze-like scenarios), where constraints are incorporated as penalty terms. This approach enables faster computation and greater adaptability, making them highly effective for real-time applications, such as obstacle avoidance or search-and-rescue operations, where computational efficiency is paramount [96,99]. Although soft-constrained methods may yield trajectories that are suboptimal in precision, their robustness in handling complex and dynamic environments makes them indispensable in such scenarios. Ultimately, selecting the appropriate method requires balancing trajectory quality, computational cost, and environmental complexity.

Overall, with the advancement of UAV autonomous navigation capabilities, UAVs have moved from controlled lab environments to the field. However, achieving safe, intelligent, efficient, and robust motion planning in complex, unknown environments remains an open challenge. Considering the current state of development, future work in UAV motion planning will likely focus on the following key areas.

Robustness: As UAV applications expand and task complexity increases, robustness in UAV motion planning becomes increasingly essential. Future UAV motion planning will necessarily involve robust planning. To enhance robustness, several factors must be addressed: environmental changes (e.g., dynamic obstacles, lighting variations, and texture degradation); and UAV-specific influences, such as airflow interference, nonlinear UAV dynamics, and the need for rapid responses during high-speed flight.

Swarm: With advancements in computing, 5G, and sensor technology, UAV communication and collaboration have become more reliable and efficient, paving the way for collaborative UAV motion planning. Swarm-based planning can significantly improve efficiency, reliability, and application range, likely driving further development in UAV technology. However, swarming also presents challenges, including group behavior control, communication and coordination, and collective obstacle avoidance.

Intelligence: With the growing impact of artificial intelligence—especially deep learning and reinforcement learning—in the field of computing, learning-based methods are expected to be a future focus in UAV motion planning. Learning-based approaches can automatically identify patterns and rules within data to develop highly efficient and robust motion planning strategies that are capable of handling complex UAV tasks and environments. However, deep learning requires substantial data and computational resources to train models, making it less suitable for real-time applications, and these models often suffer from limited generalization, making them less effective in dynamically changing scenarios.

7. Conclusions

Currently, UAVs can agilely navigate unknown and complex environments without additional human intervention, primarily thanks to advanced motion planning technologies. In this paper, we analyzed the classic works within each module of UAV motion planning, as well as recent frameworks that have been successfully implemented for real-world flight in unknown environments over the past four years. It is evident that researchers have explored various aspects of UAV autonomous navigation—such as success rate, efficiency, energy consumption, responsiveness, and environmental adaptability—and they have proposed numerous sophisticated motion planning frameworks to achieve improvements in one or more of these areas. However, considering the limited computational capacity of onboard computers and the restricted field of view and accuracy of sensors, the research outlook for UAV motion planning remains highly promising.

Author Contributions: Y.Z. (Yuquan Zhou) and L.Y. Conceptualization; Y.Z. (Yuquan Zhou) Investigation; Y.Z. (Yuquan Zhou) Writing—original draft preparation; Y.Z. (Yuquan Zhou) Writing—review and editing; Y.Z. (Yinghao Zhao) and Y.H. Supervision; L.Y. and H.X. Funding acquisition. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the National Natural Science Foundation of China (grant numbers 42394061 and 42371451) and the Natural Science Foundation of Wuhan (grant number 2024040701010028).

Data Availability Statement: There are no new data for this paper.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

Acronyms	Definitions
UAVs	Unmanned Aerial Vehicles
PCM	Point Cloud Map
OGM	Occupancy Grid Map
TSDF	Truncated Signed Distance Field
ESDF	Euclidean Signed Distance Field
SM	Semantic Map
LiDAR	Light Detection And Ranging
DFS	Depth-First Search
BFS	Breadth-First Search
D*	Dynamic A*
ARA*	Anytime A*
LPA*	Long-Life A*
Theta*	Any-Angle A*
JPS	Jump Point Search
PRM	Probabilistic Roadmap
RRT	Rapidly Exploring Random Tree
DD-RRT	Dynamic-Domain RRT
RRV	Rapidly Exploring Random Vine
RRT*	Heuristic Rapidly Exploring Random Tree
RT-RRT*	Real time-RRT*
3D	Three Dimensional
QP	Quadratic Programming
SFC	Safe Flight Corridor
RILS	Regional Inflation by Line Search
IRIS	Iterative Region Inflation by Semidefinite Programming
MISOCP	Mixed-Integer Second-Order Cone Programming

SOS	Sum of Squares
QCQP	Quadratically Constrained Quadratic Program
SOCP	Second-Order Conic Program
PCCI	Parallel Convex Cluster Inflation
UVD	Uniform Visibility Deformation
LSC	Linear Safety Corridor
SLAM	Simultaneous Localization And Mapping
GPU	Graphics Processing Unit
FIF	Fisher Information Field
MPPI	Model Predictive Path Integral
MPC	Model Predictive Contouring
MPCC	Model Predictive Contouring Control
VIO	Visual-Inertial Odometry
UWB	Ultra-Wideband

References

1. Siebert, S.; Teizer, J. Mobile 3D Mapping for Surveying Earthwork Projects Using an Unmanned Aerial Vehicle (UAV) System. *Autom. Constr.* **2014**, *41*, 1–14. [\[CrossRef\]](#)
2. Kattenborn, T.; Lopatin, J.; Förster, M.; Braun, A.C.; Fassnacht, F.E. UAV Data as Alternative to Field Sampling to Map Woody Invasive Species Based on Combined Sentinel-1 and Sentinel-2 Data. *Remote. Sens. Environ.* **2019**, *227*, 61–73. [\[CrossRef\]](#)
3. Gupta, A.; Fernando, X. Simultaneous Localization and Mapping (SLAM) and Data Fusion in Unmanned Aerial Vehicles: Recent Advances and Challenges. *Drones* **2022**, *6*, 85. [\[CrossRef\]](#)
4. Liu, J.; Wang, H.; Liu, X.; Li, F.; Sun, G.; Song, P. An Automated 3D Reconstruction Method of UAV Images. In Proceedings of the Applied Optics and Photonics China (AOPC2015), Beijing, China, 5–7 May 2015; p. 96780S. [\[CrossRef\]](#)
5. Koch, T.; Körner, M.; Fraundorfer, F. Automatic and Semantically-Aware 3D UAV Flight Planning for Image-Based 3D Reconstruction. *Remote Sens.* **2019**, *11*, 1550. [\[CrossRef\]](#)
6. Jiang, S.; Jiang, C.; Jiang, W. Efficient Structure from Motion for Large-Scale UAV Images: A Review and a Comparison of SfM Tools. *ISPRS J. Photogramm. Remote. Sens.* **2020**, *167*, 230–251. [\[CrossRef\]](#)
7. Alsamhi, S.H.; Shvetsov, A.V.; Kumar, S.; Shvetsova, S.V.; Alhartomi, M.A.; Hawbani, A.; Rajput, N.S.; Srivastava, S.; Saif, A.; Nyangaresi, V.O. UAV Computing-Assisted Search and Rescue Mission Framework for Disaster and Harsh Environment Mitigation. *Drones* **2022**, *6*, 154. [\[CrossRef\]](#)
8. Yang, T.; Jiang, Z.; Sun, R.; Cheng, N.; Feng, H. Maritime Search and Rescue Based on Group Mobile Computing for Unmanned Aerial Vehicles and Unmanned Surface Vehicles. *IEEE Trans. Ind. Inform.* **2020**, *16*, 7700–7708. [\[CrossRef\]](#)
9. Ribeiro, R.G.; Cota, L.P.; Euzebio, T.A.M.; Ramirez, J.A.; Guimaraes, F.G. Unmanned-Aerial-Vehicle Routing Problem With Mobile Charging Stations for Assisting Search and Rescue Missions in Postdisaster Scenarios. *IEEE Trans. Syst. Man Cybern. Syst.* **2022**, *52*, 6682–6696. [\[CrossRef\]](#)
10. Liu, H.; Shao, X.; Liu, J.; Zhang, Q.; Li, X. Path-Following Formation Coordination for Multiple UAVs Considering Communication Constraints. *IEEE Trans. Intell. Veh.* **2024**, *9*, 3436–3449. [\[CrossRef\]](#)
11. Radoglou-Grammatikis, P.; Sarigiannidis, P.; Lagkas, T.; Moscholios, I. A Compilation of UAV Applications for Precision Agriculture. *Comput. Netw.* **2020**, *172*, 107148. [\[CrossRef\]](#)
12. Deng, L.; Mao, Z.; Li, X.; Hu, Z.; Duan, F.; Yan, Y. UAV-based Multispectral Remote Sensing for Precision Agriculture: A Comparison between Different Cameras. *ISPRS J. Photogramm. Remote Sens.* **2018**, *146*, 124–136. [\[CrossRef\]](#)
13. Tokekar, P.; Hook, J.V.; Mulla, D.; Isler, V. Sensor Planning for a Symbiotic UAV and UGV System for Precision Agriculture. *IEEE Trans. Robot.* **2016**, *32*, 1498–1511. [\[CrossRef\]](#)
14. Hyypä, E.; Hyypä, J.; Hakala, T.; Kukko, A.; Wulder, M.A.; White, J.C.; Pyörälä, J.; Yu, X.; Wang, Y.; Virtanen, J.P.; et al. Under-Canopy UAV Laser Scanning for Accurate Forest Field Measurements. *ISPRS J. Photogramm. Remote. Sens.* **2020**, *164*, 41–60. [\[CrossRef\]](#)
15. Hyypä, E.; Yu, X.; Kaartinen, H.; Hakala, T.; Kukko, A.; Vastaranta, M.; Hyypä, J. Comparison of Backpack, Handheld, Under-Canopy UAV, and Above-Canopy UAV Laser Scanning for Field Reference Data Collection in Boreal Forests. *Remote Sens.* **2020**, *12*, 3327. [\[CrossRef\]](#)
16. Riemersma, G. Reconstruct WWII Concentration Camp Under Heavy Canopy with UAV Lidar. *Photogramm. Eng. Remote Sens.* **2020**, *86*, 265–267. [\[CrossRef\]](#)
17. Moura, A.; Antunes, J.; Dias, A.; Martins, A.; Almeida, J. Graph-SLAM Approach for Indoor UAV Localization in Warehouse Logistics Applications. In Proceedings of the 2021 IEEE International Conference on Autonomous Robot Systems and Competitions (ICARSC), Santa Maria da Feira, Portugal, 28–29 April 2021; pp. 4–11. [\[CrossRef\]](#)

18. Kang, J.; Kashihara, H.; Sumantyo, J.T.S.; Kim, J.H. Performance Analysis of Uplink NOMA Based Full-Duplex UAV for Indoor Disaster Environment. In Proceedings of the 2021 International Conference on Information and Communication Technology Convergence (ICTC), Jeju Island, Republic of Korea, 20–22 October 2021; pp. 245–247. [\[CrossRef\]](#)
19. Li, J.; Xiong, X.; Yan, Y.; Yang, Y. A Survey of Indoor UAV Obstacle Avoidance Research. *IEEE Access* **2023**, *11*, 51861–51891. [\[CrossRef\]](#)
20. Yue, X.; Shao, X.; Zhang, W. Coordinated Target Encircling Control Along a Preassigned Elliptical Orbit for Quadrotors With Aperiodic Data Transmission. *IEEE Trans. Ind. Electron.* **2024**, *71*, 16120–16131. [\[CrossRef\]](#)
21. Yan, L.; Li, Y.; Xie, H. Urban Building Mesh Polygonization Based on 1-Ring Patch and Topology Optimization. *Remote Sens.* **2021**, *13*, 4777. [\[CrossRef\]](#)
22. Yan, L.; Dai, J.; Hu, X.; Li, Y.; Su, S.; Xie, H. Automatic Targetless Extrinsic Calibration between a Spinning Actuated LiDAR and a Camera. In Proceedings of the 2022 IEEE International Conference on Unmanned Systems (ICUS), Guangzhou, China, 28–30 October 2022; pp. 303–308. [\[CrossRef\]](#)
23. Yan, L.; Li, Y.; Dai, J.; Xie, H. UBMDP: Urban Building Mesh Decoupling and Polygonization. *IEEE Trans. Geosci. Remote. Sens.* **2023**, *61*, 1–16. [\[CrossRef\]](#)
24. Zhang, F.; Shao, X.; Zhang, W. Cooperative Fusion Localization of a Nonstationary Target for Multiple UAVs Without GPS. *IEEE Syst. J.* **2024**, *18*, 1951–1962. [\[CrossRef\]](#)
25. Cao, C.; Zhu, H.; Choset, H.; Zhang, J. Exploring Large and Complex Environments Fast and Efficiently. In Proceedings of the 2021 IEEE International Conference on Robotics and Automation (ICRA), Xi'an, China, 30 May–5 June 2021; pp. 7781–7787. [\[CrossRef\]](#)
26. Cao, C.; Zhu, H.; Ren, Z.; Choset, H.; Zhang, J. Representation Granularity Enables Time-Efficient Autonomous Exploration in Large, Complex Worlds. *Sci. Robot.* **2023**, *8*, eadf0970. [\[CrossRef\]](#)
27. Zhao, Y.; Yan, L.; Xie, H.; Dai, J.; Wei, P. Autonomous Exploration Method for Fast Unknown Environment Mapping by Using UAV Equipped with Limited FOV Sensor. *IEEE Trans. Ind. Electron.* **2023**, *71*, 4933–4943. [\[CrossRef\]](#)
28. Qin, H.; Shao, S.; Wang, T.; Yu, X.; Jiang, Y.; Cao, Z. Review of Autonomous Path Planning Algorithms for Mobile Robots. *Drones* **2023**, *7*, 211. [\[CrossRef\]](#)
29. Aggarwal, S.; Kumar, N. Path Planning Techniques for Unmanned Aerial Vehicles: A Review, Solutions, and Challenges. *Comput. Commun.* **2020**, *149*, 270–299. [\[CrossRef\]](#)
30. Luo, J.; Tian, Y.; Wang, Z. Research on Unmanned Aerial Vehicle Path Planning. *Drones* **2024**, *8*, 51. [\[CrossRef\]](#)
31. Yahia, H.S.; Mohammed, A.S. Path Planning Optimization in Unmanned Aerial Vehicles Using Meta-Heuristic Algorithms: A Systematic Review. *Environ. Monit. Assess.* **2023**, *195*, 30. [\[CrossRef\]](#)
32. Ait Saadi, A.; Soukane, A.; Meraihi, Y.; Benmessaoud Gabis, A.; Mirjalili, S.; Ramdane-Cherif, A. UAV Path Planning Using Optimization Approaches: A Survey. *Arch. Comput. Methods Eng.* **2022**, *29*, 4233–4284. [\[CrossRef\]](#)
33. Fang, Z.; Savkin, A.V. Strategies for Optimized UAV Surveillance in Various Tasks and Scenarios: A Review. *Drones* **2024**, *8*, 193. [\[CrossRef\]](#)
34. Gao, F.; Shen, S. Online Quadrotor Trajectory Generation and Autonomous Navigation on Point Clouds. In Proceedings of the 2016 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR), Lausanne, Switzerland, 23–27 October 2016; pp. 139–146. [\[CrossRef\]](#)
35. Elfes, A. Using Occupancy Grids for Mobile Robot Perception and Navigation. *Computer* **1989**, *22*, 46–57. [\[CrossRef\]](#)
36. Hornung, A.; Wurm, K.M.; Bennewitz, M.; Stachniss, C.; Burgard, W. OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees. *Auton. Robot.* **2013**, *34*, 189–206. [\[CrossRef\]](#)
37. Newcombe, R.A.; Fitzgibbon, A.; Izadi, S.; Hilliges, O.; Molyneaux, D.; Kim, D.; Davison, A.J.; Kohi, P.; Shotton, J.; Hodges, S. KinectFusion: Real-time Dense Surface Mapping and Tracking. In Proceedings of the 2011 10th IEEE International Symposium on Mixed and Augmented Reality, Basel, Switzerland, 26–29 October 2011; pp. 127–136. [\[CrossRef\]](#)
38. Curless, B.; Levoy, M. A Volumetric Method for Building Complex Models from Range Images. In Proceedings of the Proceedings of the 23rd Annual Conference on Computer Graphics and Interactive Techniques—SIGGRAPH '96, New Orleans, LA, USA, 4–9 August 1996; pp. 303–312. [\[CrossRef\]](#)
39. Chen, S.W.; Nardari, G.V.; Lee, E.S.; Qu, C.; Liu, X.; Romero, R.A.F.; Kumar, V. SLOAM: Semantic Lidar Odometry and Mapping for Forest Inventory. *IEEE Robot. Autom. Lett.* **2020**, *5*, 612–619. [\[CrossRef\]](#)
40. Duberg, D.; Jensfelt, P. UFOMap: An Efficient Probabilistic 3D Mapping Framework That Embraces the Unknown. *IEEE Robot. Autom. Lett.* **2020**, *5*, 6411–6418. [\[CrossRef\]](#)
41. Lin, Y.; Gao, F.; Qin, T.; Gao, W.; Liu, T.; Wu, W.; Yang, Z.; Shen, S. Autonomous Aerial Navigation Using Monocular Visual-Inertial Fusion. *J. Field Robot.* **2018**, *35*, 23–51. [\[CrossRef\]](#)
42. Han, L.; Gao, F.; Zhou, B.; Shen, S. FIESTA: Fast Incremental Euclidean Distance Fields for Online Motion Planning of Aerial Robots. In Proceedings of the 2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Macau, China, 4–8 November 2019; pp. 4423–4430. [\[CrossRef\]](#)

43. Tao, Y.; Liu, X.; Spasojevic, I.; Agarwal, S.; Kumar, V. 3D Active Metric-Semantic SLAM. *IEEE Robot. Autom. Lett.* **2024**, *9*, 2989–2996. [\[CrossRef\]](#)
44. Rusu, R.B.; Cousins, S. 3d Is Here: Point Cloud Library (Pcl). In Proceedings of the 2011 IEEE International Conference on Robotics and Automation, Shanghai, China, 9–13 May 2011; pp. 1–4.
45. Zhou, K.; Hou, Q.; Wang, R.; Guo, B. Real-Time KD-tree Construction on Graphics Hardware. *ACM Trans. Graph.* **2008**, *27*, 1–11. [\[CrossRef\]](#)
46. Yan, Z.; Wu, X.; Jian, Z.; Lan, B.; Wang, X. RH-Map: Online Map Construction Framework of Dynamic Object Removal Based on 3D Region-wise Hash Map Structure. *IEEE Robot. Autom. Lett.* **2023**, *9*, 1492–1499. [\[CrossRef\]](#)
47. Ren, Y.; Cai, Y.; Zhu, F.; Liang, S.; Zhang, F. ROG-map: An Efficient Robocentric Occupancy Grid Map for Large-Scene and High-Resolution LiDAR-based Motion Planning. *arXiv* **2023**, arXiv:2302.14819. [\[2302.14819\]](#).
48. Meagher, D. Geometric Modeling Using Octree Encoding. *Comput. Graph. Image Process.* **1982**, *19*, 129–147. [\[CrossRef\]](#)
49. Usenko, V.; von Stumberg, L.; Pangercic, A.; Cremers, D. Real-Time Trajectory Replanning for MAVs Using Uniform B-splines and a 3D Circular Buffer. In Proceedings of the 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Vancouver, BC, Canada, 24–28 September 2017; pp. 215–222. [\[CrossRef\]](#)
50. Jia, T.; Yang, E.Y.; Hsiao, Y.S.; Cruz, J.; Brooks, D.; Wei, G.Y.; Reddi, V.J. OMU: A Probabilistic 3D Occupancy Mapping Accelerator for Real-time OctoMap at the Edge. In Proceedings of the 2022 Design, Automation & Test in Europe Conference & Exhibition (DATE), Antwerp, Belgium, 14–23 March 2022; pp. 909–914. [\[CrossRef\]](#)
51. Ratliff, N.; Zucker, M.; Bagnell, J.A.; Srinivasa, S. CHOMP: Gradient Optimization Techniques for Efficient Motion Planning. In Proceedings of the 2009 IEEE International Conference on Robotics and Automation, Kobe, Japan, 12–17 May 2009; pp. 489–494. [\[CrossRef\]](#)
52. Oleynikova, H.; Taylor, Z.; Fehr, M.; Siegwart, R.; Nieto, J. Voxblox: Incremental 3D Euclidean Signed Distance Fields for on-Board MAV Planning. In Proceedings of the 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Vancouver, BC, Canada, 24–28 September 2017; pp. 1366–1373. [\[CrossRef\]](#)
53. Geng, S.; Wang, Q.; Xie, L.; Xu, C.; Cao, Y.; Gao, F. Robo-Centric ESDF: A Fast and Accurate Whole-Body Collision Evaluation Tool for Any-Shape Robotic Planning. In Proceedings of the 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Detroit, MI, USA, 1–5 October 2023; pp. 290–297.
54. Bai, Y.; Miao, Z.; Wang, X.; Liu, Y.; Wang, H.; Wang, Y. Vdbblox: Accurate and Efficient Distance Fields for Path Planning and Mesh Reconstruction. In Proceedings of the 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Detroit, MI, USA, 1–5 October 2023; pp. 7187–7194.
55. Liu, X.; Nardari, G.V.; Ojeda, F.C.; Tao, Y.; Zhou, A.; Donnelly, T.; Qu, C.; Chen, S.W.; Romero, R.A.F.; Taylor, C.J.; et al. Large-Scale Autonomous Flight With Real-Time Semantic SLAM Under Dense Forest Canopy. *IEEE Robot. Autom. Lett.* **2022**, *7*, 5512–5519. [\[CrossRef\]](#)
56. Cormen, T.H.; Leiserson, C.E.; Rivest, R.L.; Stein, C. *Introduction to Algorithms*; MIT Press: Cambridge, MA, USA, 2022.
57. Dijkstra, E.W. A Note on Two Problems in Connexion with Graphs. *Numer. Math.* **1959**, *1*, 269–271. [\[CrossRef\]](#)
58. A Comprehensive Path-Finding Library in Javascript. Available online: <https://qiao.github.io/PathFinding.js/visual/> (accessed on 9 January 2025).
59. Hart, P.; Nilsson, N.; Raphael, B. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Trans. Syst. Sci. Cybern.* **1968**, *4*, 100–107. [\[CrossRef\]](#)
60. Stentz, A. Optimal and Efficient Path Planning for Partially Known Environments. In Proceedings of the 1994 IEEE International Conference on Robotics and Automation, San Diego, CA, USA, 8–13 May 1994; pp. 203–220.
61. Likhachev, M.; Gordon, G.J.; Thrun, S. ARA*ast: Anytime A*ast with Provable Bounds on Sub-Optimality. In *Advances in Neural Information Processing Systems*; Thrun, S.; Saul, L.; Schölkopf, B., Eds.; MIT Press: Cambridge, MA, USA, 2003; Volume 16.
62. Koenig, S.; Likhachev, M.; Furcy, D. Lifelong Planning A*. *Artif. Intell.* **2004**, *155*, 93–146. [\[CrossRef\]](#)
63. Daniel, K.; Nash, A.; Koenig, S.; Felner, A. Theta*: Any-Angle Path Planning on Grids. *J. Artif. Intell. Res.* **2010**, *39*, 533–579. [\[CrossRef\]](#)
64. Yap, P.; Burch, N.; Holte, R.; Schaeffer, J. Block A*: Database-Driven Search with Applications in Any-Angle Path-Planning. *Proc. AAAI Conf. Artif. Intell.* **2011**, *25*, 120–125. [\[CrossRef\]](#)
65. Harabor, D.; Grastien, A. Online Graph Pruning for Pathfinding On Grid Maps. *Proc. AAAI Conf. Artif. Intell.* **2011**, *25*, 1114–1119. [\[CrossRef\]](#)
66. Dolgov, D.; Thrun, S.; Montemerlo, M.; Diebel, J. Path Planning for Autonomous Vehicles in Unknown Semi-structured Environments. *Int. J. Robot. Res.* **2010**, *29*, 485–501. [\[CrossRef\]](#)
67. Pivtoraiko, M.; Kelly, A. Generating near Minimal Spanning Control Sets for Constrained Motion Planning in Discrete State Spaces. In Proceedings of the 2005 IEEE/RSJ International Conference on Intelligent Robots and Systems, Edmonton, AB, Canada, 2–6 August 2005; pp. 3231–3237.

68. Kavraki, L.; Svestka, P.; Latombe, J.C.; Overmars, M. Probabilistic Roadmaps for Path Planning in High-Dimensional Configuration Spaces. *IEEE Trans. Robot. Autom.* **1996**, *12*, 566–580. [\[CrossRef\]](#)
69. LaValle, S.M. *Rapidly-Exploring Random Trees: A New Tool for Path Planning*; Research Report; Iowa State University: Ames, IA, USA, 1998.
70. Kuffner, J.; LaValle, S. RRT-connect: An Efficient Approach to Single-Query Path Planning. In Proceedings of the Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No.00CH37065), San Francisco, CA, USA, 24–28 April 2000; Volume 2, pp. 995–1001. [\[CrossRef\]](#)
71. Yershova, A.; Jaillet, L.; Siméon, T.; LaValle, S.M. Dynamic-Domain RRTs: Efficient Exploration by Controlling the Sampling Domain. In Proceedings of the 2005 IEEE International Conference on Robotics and Automation, Barcelona, Spain, 18–22 April 2005; pp. 3856–3861.
72. Tahirovic, A.; Ferizbegovic, M. Rapidly-Exploring Random Vines (RRV) for Motion Planning in Configuration Spaces with Narrow Passages. In Proceedings of the 2018 IEEE International Conference on Robotics and Automation (ICRA), Brisbane, QLD, Australia, 21–25 May 2018; pp. 7055–7062. [\[CrossRef\]](#)
73. Karaman, S.; Frazzoli, E. Incremental Sampling-based Algorithms for Optimal Motion Planning. *Robot. Sci. Syst.* **2010**, *104*, 267–274. [\[CrossRef\]](#)
74. Islam, F.; Nasir, J.; Malik, U.; Ayaz, Y.; Hasan, O. RRT*-Smart: Rapid Convergence Implementation of RRT* towards Optimal Solution. In Proceedings of the 2012 IEEE International Conference on Mechatronics and Automation, Chengdu, China, 5–8 August 2012; pp. 1651–1656. [\[CrossRef\]](#)
75. Naderi, K.; Rajamäki, J.; Härmäläinen, P. RT-RRT*: A Real-Time Path Planning Algorithm Based on RRT*. In Proceedings of the 8th ACM SIGGRAPH Conference on Motion in Games, Paris, France, 16–18 November 2015; pp. 113–118. [\[CrossRef\]](#)
76. Gammell, J.D.; Barfoot, T.D.; Srinivasa, S.S. Informed Sampling for Asymptotically Optimal Path Planning. *IEEE Trans. Robot.* **2018**, *34*, 966–984. [\[CrossRef\]](#)
77. Webb, D.J.; van den Berg, J. Kinodynamic RRT*: Asymptotically Optimal Motion Planning for Robots with Linear Dynamics. In Proceedings of the 2013 IEEE International Conference on Robotics and Automation, Karlsruhe, Germany, 6–10 May 2013; pp. 5054–5061. [\[CrossRef\]](#)
78. Mellinger, D.; Kumar, V. Minimum Snap Trajectory Generation and Control for Quadrotors. In Proceedings of the 2011 IEEE International Conference on Robotics and Automation, Shanghai, China, 9–13 May 2011; pp. 2520–2525. [\[CrossRef\]](#)
79. Richter, C.; Bry, A.; Roy, N. Polynomial Trajectory Planning for Aggressive Quadrotor Flight in Dense Indoor Environments. In *Robotics Research*; Inaba, M.; Corke, P., Eds.; Springer International Publishing: Cham, Switzerland, 2016; Volume 114, pp. 649–666. [\[CrossRef\]](#)
80. Deits, R.; Tedrake, R. Efficient Mixed-Integer Planning for UAVs in Cluttered Environments. In Proceedings of the 2015 IEEE International Conference on Robotics and Automation (ICRA), Seattle, WA, USA, 26–30 May 2015; pp. 42–49. [\[CrossRef\]](#)
81. Deits, R.; Tedrake, R. Computing Large Convex Regions of Obstacle-Free Space through Semidefinite Programming. In *Proceedings of the Algorithmic Foundations of Robotics XI: Selected Contributions of the Eleventh International Workshop on the Algorithmic Foundations of Robotics*; Springer: Berlin/Heidelberg, Germany, 2015; pp. 109–124.
82. Sikang Liu.; Watterson, M.; Tang, S.; Kumar, V. High Speed Navigation For Quadrotors With Limited Onboard Sensing. In Proceedings of the 2016 IEEE International Conference on Robotics and Automation (ICRA), Stockholm, Sweden, 16–21 May 2016; pp. 1484–1491. [\[CrossRef\]](#)
83. Chen, J.; Liu, T.; Shen, S. Online Generation of Collision-Free Trajectories for Quadrotor Flight in Unknown Cluttered Environments. In Proceedings of the 2016 IEEE International Conference on Robotics and Automation (ICRA), Stockholm, Sweden, 16–21 May 2016; pp. 1476–1483. [\[CrossRef\]](#)
84. Liu, S.; Watterson, M.; Mohta, K.; Sun, K.; Bhattacharya, S.; Taylor, C.J.; Kumar, V. Planning Dynamically Feasible Trajectories for Quadrotors Using Safe Flight Corridors in 3-D Complex Environments. *IEEE Robot. Autom. Lett.* **2017**, *2*, 1688–1695. [\[CrossRef\]](#)
85. Gao, F.; Wu, W.; Lin, Y.; Shen, S. Online Safe Trajectory Generation for Quadrotors Using Fast Marching Method and Bernstein Basis Polynomial. In Proceedings of the 2018 IEEE International Conference on Robotics and Automation (ICRA), Brisbane, QLD, Australia, 21–25 May 2018; pp. 344–351. [\[CrossRef\]](#)
86. Farin, G. Algorithms for Rational Bézier Curves. *Comput.-Aided Des.* **1983**, *15*, 73–77. [\[CrossRef\]](#)
87. Gao, F.; Wu, W.; Gao, W.; Shen, S. Flying on Point Clouds: Online Trajectory Generation and Autonomous Navigation for Quadrotors in Cluttered Environments. *J. Field Robot.* **2019**, *36*, 710–733. [\[CrossRef\]](#)
88. Tordesillas, J.; Lopez, B.T.; How, J.P. FASTER: Fast and Safe Trajectory Planner for Flights in Unknown Environments. In Proceedings of the 2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Macau, China, 4–8 November 2019; pp. 1934–1940. [\[CrossRef\]](#)
89. Gao, F.; Wang, L.; Zhou, B.; Zhou, X.; Pan, J.; Shen, S. Teach-Repeat-Replan: A Complete and Robust System for Aggressive Flight in Complex Environments. *IEEE Trans. Robot.* **2020**, *36*, 1526–1545. [\[CrossRef\]](#)

90. Verschueren, D.; Demeulenaere, B.; Swevers, J.; Schutter, J.D.; Diehl, M. Time-Optimal Path Tracking for Robots: A Convex Optimization Approach. *IEEE Trans. Autom. Control.* **2009**, *54*, 2318–2327. [\[CrossRef\]](#)
91. Kalakrishnan, M.; Chitta, S.; Theodorou, E.; Pastor, P.; Schaal, S. STOMP: Stochastic Trajectory Optimization for Motion Planning. In Proceedings of the 2011 IEEE International Conference on Robotics and Automation, Shanghai, China, 9–13 May 2011; pp. 4569–4574. [\[CrossRef\]](#)
92. Oleynikova, H.; Burri, M.; Taylor, Z.; Nieto, J.; Siegwart, R.; Galceran, E. Continuous-Time Trajectory Optimization for Online UAV Replanning. In Proceedings of the 2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Daejeon, Republic of Korea, 9–14 October 2016; pp. 5332–5339. [\[CrossRef\]](#)
93. Burri, M.; Oleynikova, H.; Achtelik, M.W.; Siegwart, R. Real-Time Visual-Inertial Mapping, Re-Localization and Planning Onboard MAVs in Unknown Environments. In Proceedings of the 2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Hamburg, Germany, 28 September–2 October 2015; pp. 1872–1878. [\[CrossRef\]](#)
94. Eilers, P.H.; Marx, B.D. Flexible Smoothing with B-splines and Penalties. *Stat. Sci.* **1996**, *11*, 89–121. [\[CrossRef\]](#)
95. Gao, F.; Lin, Y.; Shen, S. Gradient-Based Online Safe Trajectory Generation for Quadrotor Flight in Complex Environments. In Proceedings of the 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Vancouver, BC, Canada, 24–28 September 2017; pp. 3681–3688. [\[CrossRef\]](#)
96. Zhou, B.; Gao, F.; Wang, L.; Liu, C.; Shen, S. Robust and Efficient Quadrotor Trajectory Generation for Fast Autonomous Flight. *IEEE Robot. Autom. Lett.* **2019**, *4*, 3529–3536. [\[CrossRef\]](#)
97. Zhou, B.; Gao, F.; Pan, J.; Shen, S. Robust Real-time UAV Replanning Using Guided Gradient-based Optimization and Topological Paths. In Proceedings of the 2020 IEEE International Conference on Robotics and Automation (ICRA), Paris, France, 31 May–31 August 2020; pp. 1208–1214. [\[CrossRef\]](#)
98. Jaillet, L.; Siméon, T. Path Deformation Roadmaps: Compact Graphs with Useful Cycles for Motion Planning. *Int. J. Robot. Res.* **2008**, *27*, 1175–1188. [\[CrossRef\]](#)
99. Zhou, X.; Wang, Z.; Ye, H.; Xu, C.; Gao, F. EGO-Planner: An ESDF-Free Gradient-Based Local Planner for Quadrotors. *IEEE Robot. Autom. Lett.* **2021**, *6*, 478–485. [\[CrossRef\]](#)
100. Kong, F.; Xu, W.; Cai, Y.; Zhang, F. Avoiding Dynamic Small Obstacles with Onboard Sensing and Computation on Aerial Robots. *IEEE Robot. Autom. Lett.* **2021**, *6*, 7869–7876. [\[CrossRef\]](#)
101. Zhou, B.; Pan, J.; Gao, F.; Shen, S. RAPTOR: Robust and Perception-Aware Trajectory Replanning for Quadrotor Fast Flight. *IEEE Trans. Robot.* **2021**, *37*, 1992–2009. [\[CrossRef\]](#)
102. Ji, J.; Wang, Z.; Wang, Y.; Xu, C.; Gao, F. Mapless-Planner: A Robust and Fast Planning Framework for Aggressive Autonomous Flight without Map Fusion. In Proceedings of the 2021 IEEE International Conference on Robotics and Automation (ICRA), Xi'an, China, 30 May–5 June 2021; pp. 6315–6321.
103. Zhao, Y.; Yan, L.; Chen, Y.; Dai, J.; Liu, Y. Robust and Efficient Trajectory Replanning Based on Guiding Path for Quadrotor Fast Autonomous Flight. *Remote Sens.* **2021**, *13*, 972. [\[CrossRef\]](#)
104. Ye, H.; Zhou, X.; Wang, Z.; Xu, C.; Chu, J.; Gao, F. TGK-Planner: An Efficient Topology Guided Kinodynamic Planner for Autonomous Quadrotors. *IEEE Robot. Autom. Lett.* **2021**, *6*, 494–501. [\[CrossRef\]](#)
105. Quan, L.; Zhang, Z.; Zhong, X.; Xu, C.; Gao, F. EVA-Planner: Environmental Adaptive Quadrotor Planning. In Proceedings of the 2021 IEEE International Conference on Robotics and Automation (ICRA), Xi'an, China, 30 May–5 June 2021; pp. 398–404. [\[CrossRef\]](#)
106. Wang, L.; Zhou, B.; Liu, C.; Shen, S. Estimation and Adaption of Indoor Ego Airflow Disturbance with Application to Quadrotor Trajectory Planning. In Proceedings of the 2021 IEEE International Conference on Robotics and Automation (ICRA), Xi'an, China, 30 May–5 June 2021; pp. 384–390. [\[CrossRef\]](#)
107. Zhou, X.; Zhu, J.; Zhou, H.; Xu, C.; Gao, F. EGO-Swarm: A Fully Autonomous and Decentralized Quadrotor Swarm System in Cluttered Environments. In Proceedings of the 2021 IEEE International Conference on Robotics and Automation (ICRA), Xi'an, China, 30 May–5 June 2021; pp. 4101–4107. [\[CrossRef\]](#)
108. Wang, Y.; Ji, J.; Wang, Q.; Xu, C.; Gao, F. Autonomous Flights in Dynamic Environments with Onboard Vision. In Proceedings of the 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Prague, Czech Republic, 27 September–1 October 2021; pp. 1966–1973. [\[CrossRef\]](#)
109. Sun, W.; Tang, G.; Hauser, K. Fast UAV Trajectory Optimization Using Bilevel Optimization with Analytical Gradients. *IEEE Trans. Robot.* **2021**, *37*, 2010–2024. [\[CrossRef\]](#)
110. Schleich, D.; Beul, M.; Quenzel, J.; Behnke, S. Autonomous Flight in Unknown GNSS-denied Environments for Disaster Examination. In Proceedings of the 2021 International Conference on Unmanned Aircraft Systems (ICUAS), Athens, Greece, 15–18 June 2021; pp. 950–957.
111. Wang, Z.; Zhou, X.; Xu, C.; Gao, F. Geometrically Constrained Trajectory Optimization for Multicopters. *IEEE Trans. Robot.* **2022**, *38*, 3259–3278. [\[CrossRef\]](#)

112. Zhou, X.; Wen, X.; Wang, Z.; Gao, Y.; Li, H.; Wang, Q.; Yang, T.; Lu, H.; Cao, Y.; Xu, C.; et al. Swarm of Micro Flying Robots in the Wild. *Sci. Robot.* **2022**, *7*, eabm5954. [\[CrossRef\]](#)
113. Quan, L.; Yin, L.; Xu, C.; Gao, F. Distributed Swarm Trajectory Optimization for Formation Flight in Dense Environments. In Proceedings of the 2022 International Conference on Robotics and Automation (ICRA), Philadelphia, PA, USA, 23–27 May 2022; pp. 4979–4985. [\[CrossRef\]](#)
114. Guo, J.; Xun, Z.; Geng, S.; Lin, Y.; Xu, C.; Gao, F. Dynamic Free-Space Roadmap for Safe Quadrotor Motion Planning. In Proceedings of the 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Kyoto, Japan, 23–27 October 2022; pp. 10523–10528. [\[CrossRef\]](#)
115. Park, J.; Kim, D.; Kim, G.C.; Oh, D.; Kim, H.J. Online Distributed Trajectory Planning for Quadrotor Swarm with Feasibility Guarantee Using Linear Safe Corridor. *IEEE Robot. Autom. Lett.* **2022**, *7*, 4869–4876. [\[CrossRef\]](#)
116. Ren, Y.; Zhu, F.; Liu, W.; Wang, Z.; Lin, Y.; Gao, F.; Zhang, F. Bubble Planner: Planning High-speed Smooth Quadrotor Trajectories Using Receding Corridors. In Proceedings of the 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Kyoto, Japan, 23–27 October 2022; pp. 6332–6339. [\[CrossRef\]](#)
117. Yan, L.; Zhao, Y.; Xie, H.; Dai, J.; Han, Y.; Su, S. An Inert and Efficient Fusion Replanning Method for UAV Fast Autonomous Flight. In Proceedings of the 2022 IEEE International Conference on Unmanned Systems (ICUS), Guangzhou, China, 28–30 October 2022; pp. 140–145. [\[CrossRef\]](#)
118. Li, H.; Wang, H.; Feng, C.; Gao, F.; Zhou, B.; Shen, S. AutoTrans: A Complete Planning and Control Framework for Autonomous UAV Payload Transportation. *IEEE Robot. Autom. Lett.* **2023**, *8*, 6859–6866. [\[CrossRef\]](#)
119. Cao, M.; Cao, K.; Yuan, S.; Nguyen, T.M.; Xie, L. Neptune: Nonentangling Trajectory Planning for Multiple Tethered Unmanned Vehicles. *IEEE Trans. Robot.* **2023**, *39*, 2786–2804. [\[CrossRef\]](#)
120. Wang, L.; Guo, Y. Speed Adaptive Robot Trajectory Generation Based on Derivative Property of B-Spline Curve. *IEEE Robot. Autom. Lett.* **2023**, *8*, 1905–1911. [\[CrossRef\]](#)
121. Ma, Z.; Wang, Z.; Ma, A.; Liu, Y.; Niu, Y. A Low-Altitude Obstacle Avoidance Method for UAVs Based on Polyhedral Flight Corridor. *Drones* **2023**, *7*, 588. [\[CrossRef\]](#)
122. Quan, L.; Yin, L.; Zhang, T.; Wang, M.; Wang, R.; Zhong, S.; Zhou, X.; Cao, Y.; Xu, C.; Gao, F. Robust and Efficient Trajectory Planning for Formation Flight in Dense Environments. *IEEE Trans. Robot.* **2023**, *39*, 4785–4804. [\[CrossRef\]](#)
123. Liu, S.; Guo, K.; Yu, X.; Ma, L.; Xie, L.; Guo, L. Safe Maneuvering Planning for Flights in Complex Environments. *IEEE Trans. Ind. Electron.* **2023**, *71*, 4944–4953. [\[CrossRef\]](#)
124. Gao, J.; He, F.; Zhang, W.; Yao, Y. Obstacle-Aware Topological Planning over Polyhedral Representation for Quadrotors. In Proceedings of the 2023 IEEE International Conference on Robotics and Automation (ICRA), London, UK, 29 May–2 June 2023; pp. 10097–10103.
125. Wang, H.; Zhang, X.; Liu, Y.; Zhang, X.; Zhuang, Y. Svpto: Safe Visibility-Guided Perception-Aware Trajectory Optimization for Aerial Tracking. *IEEE Trans. Ind. Electron.* **2023**, *71*, 2716–2725. [\[CrossRef\]](#)
126. Chen, H.; Wen, C.Y.; Gao, F.; Lu, P. Flying in Dynamic Scenes With Multitarget Velocimetry and Perception-Enhanced Planning. *IEEE/ASME Trans. Mechatronics* **2023**, *29*, 521–532. [\[CrossRef\]](#)
127. Liu, T.; Zhang, F.; Gao, F.; Pan, J. Tight Collision Probability for Uav Motion Planning in Uncertain Environment. In Proceedings of the 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Detroit, MI, USA, 1–5 October 2023; pp. 1055–1062.
128. Chen, G.; Peng, P.; Zhang, P.; Dong, W. Risk-Aware Trajectory Sampling for Quadrotor Obstacle Avoidance in Dynamic Environments. *IEEE Trans. Ind. Electron.* **2023**, *70*, 12606–12615. [\[CrossRef\]](#)
129. Liu, W.; Ren, Y.; Zhang, F. Integrated Planning and Control for Quadrotor Navigation in Presence of Suddenly Appearing Objects and Disturbances. *IEEE Robot. Autom. Lett.* **2023**, *9*, 899–906. [\[CrossRef\]](#)
130. Liang, X.; Yao, H.; Qi, H.; Wang, X. Forest in Situ Observations through a Fully Automated Under-Canopy Unmanned Aerial Vehicle. *Geo-Spat. Inf. Sci.* **2024**, *27*, 983–999. [\[CrossRef\]](#)
131. Zhong, J.; Li, M.; Chen, Y.; Wei, Z.; Yang, F.; Shen, H. A Safer Vision-Based Autonomous Planning System for Quadrotor Uavs with Dynamic Obstacle Trajectory Prediction and Its Application with Llms. In Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision, Waikoloa, HI, USA, 3–8 January 2024; pp. 920–929.
132. Zhang, X.; Tian, B.; Lu, H.; Shen, H.; Lu, J. PAST: A Decentralized Multi-MAV Online Navigation System Using Parallel Strategy Acceleration. In *IEEE Transactions on Intelligent Vehicles*; IEEE: New York, NY, USA, 2024.
133. Sun, G.; Zhang, X.; Liu, Y.; Zhang, X.; Zhuang, Y. Safety-Driven and Localization Uncertainty-Driven Perception-Aware Trajectory Planning for Quadrotor Unmanned Aerial Vehicles. *IEEE Trans. Intell. Transp. Syst.* **2024**, *25*, 8837–8848. [\[CrossRef\]](#)
134. Qiu, J.; Liu, Q.; Qin, J.; Cheng, D.; Tian, Y.; Ma, Q. PE-Planner: A Performance-Enhanced Quadrotor Motion Planner for Autonomous Flight in Complex and Dynamic Environments. *IEEE Robot. Autom. Lett.* **2024**, *9*, 5879–5886. [\[CrossRef\]](#)
135. Shi, L.; Jiang, W.; Luo, Z.; Yang, L.; Wang, Y. Enhancing Adaptability: Hierarchical Frontier-based Path Planning for Navigation in Challenging Environments. *IEEE Robot. Autom. Lett.* **2024**, *9*, 8611–8618. [\[CrossRef\]](#)

136. Yang, C.; Zhou, H.; Liu, X.; Ke, Y.; Gao, B.; Grzegorzec, M.; Boukhers, Z.; Chen, T.; See, J. BladeView: Toward Automatic Wind Turbine Inspection With Unmanned Aerial Vehicle. In *IEEE Transactions on Automation Science and Engineering*; IEEE: New York, NY, USA, 2024.
137. Tordesillas, J.; How, J.P. MINVO Basis: Finding Simplexes with Minimum Volume Enclosing Polynomial Curves. *Comput.-Aided Des.* **2022**, *151*, 103341. [[CrossRef](#)]
138. Tordesillas, J.; Lopez, B.T.; Everett, M.; How, J.P. FASTER: Fast and Safe Trajectory Planner for Navigation in Unknown Environments. *IEEE Trans. Robot.* **2022**, *38*, 922–938. [[CrossRef](#)]
139. Ren, Y.; Zhu, F.; Lu, G.; Cai, Y.; Yin, L.; Kong, F.; Lin, J.; Chen, N.; Zhang, F. Safety-Assured High-Speed Navigation for MAVs. *Sci. Robot.* **2025**, *10*, eado6187. [[CrossRef](#)] [[PubMed](#)]
140. Zhou, B.; Zhang, Y.; Chen, X.; Shen, S. FUEL: Fast UAV Exploration Using Incremental Frontier Structure and Hierarchical Planning. *IEEE Robot. Autom. Lett.* **2021**, *6*, 779–786. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.